Week 12 — Linux I/O and Readiness

Format: seven daily sections collected into one weekly module

Suggested workload: approximately 21 focused hours, adjustable to mastery

> A descriptor is a small integer so user code never holds a kernel address. The VFS makes different objects answer the same calls. Blocking and readiness exist so a task can leave the CPU until one of those objects can move bytes. The page cache bridges that byte API to pages and blocks.

The lab is projects/linux-io. Wrappers build through its Makefile and print only the claim the run supports. Kernel names below were read from tree `c9b2385d6793`, whose top-level `Makefile` sets `VERSION = 5`, `PATCHLEVEL = 10`, `SUBLEVEL = 134`. Those names are that snapshot. The host that runs the lab may be another kernel; the wrappers print `uname`. Re-open the pinned tree, or the sources for the kernel you boot, before treating a symbol, field, or errno path as current.

Why these seven days belong together

User space asks for bytes. The kernel keeps objects, pages, and wait queues, and the descriptor is the integer boundary between them.

```
descriptor
  fd table -> open-file description (offset, flags, refcount) -> file
        |
        v
VFS
  one read / write / stat shape; the object supplies the operations
        |
        v
bytes and waiting
  short counts, EOF, EINTR, EAGAIN
  a task sleeps, or select/poll/epoll reports that progress is possible
        |
        v
page cache
  buffered write is a cache update; fsync/msync is a request, not a crash proof
  MAP_SHARED names those same pages
        |
        v
pipes
  a finite buffer, EOF only when every writer is gone, PIPE_BUF atomicity
        |
        v
evidence
  /proc/self/fd, EBADF, EPIPE, an epoll errno, optional strace
```

In the excerpt tree, `fs/file.c` `dup` / `do_dup2` takes another reference with `get_file` and stores that same `struct file` in a second fd slot. `fs/open.c` `do_sys_open` is the path that allocates a new open. `fs/read_write.c` `vfs_read` / `vfs_write` are the common entry. `fs/sync.c` `vfs_fsync` is the durability request. `mm/filemap.c` `generic_file_write_iter` and

`filemap_fault` are the page-cache side of write and of a mapped fault. `fs/pipe.c` `pipe_write` returns `-EPIPE` when no readers remain, and `pipe_read` stops when no writers remain. `fs/select.c` implements `select` and `poll`. `fs/eventpoll.c` defines `EPOLLET` and wakes through `ep_poll_callback`. Quote those only from the tree you opened.

Daily roadmap

Day 1 — Descriptors, open files, and offsets

Day 1 splits three layers: the integer in the process, the open-file description, and the file object. `dup` shares the description, so a read or `lseek` on either descriptor moves one offset. A second `open` of the same path allocates another description, and the lab reads the leading bytes again. Wrapper: challenges/day-001-descriptors.sh.

Day 2 — Read, write, short I/O, and nonblocking

Day 2 treats a positive short count as success for those bytes. A zero read on a pipe whose writers are gone is EOF. `EAGAIN` is the nonblocking empty result. `EINTR`, with `SA_RESTART` off, is a signal interrupting the wait; the retry loop tries again. The lab forces each one: a pipe capacity smaller than the write, a closed writer, `O_NONBLOCK`, and `SIGUSR1` delivered while `read` is the current syscall. Wrapper: challenges/day-002-stream-io.sh.

Day 3 — VFS, the page cache, and durability

Day 3 starts from the VFS question: one `read` / `write` / `stat` shape, many objects. For a regular file the page cache is the memory of those bytes. The lab writes, reads the bytes back through a second descriptor before `fsync`, then calls `fsync` and records that the call returned. A sparse file has a logical `st_size` past a hole, a zero byte in that hole, and fewer allocated blocks than the logical size. None of that is a claim that power loss keeps the data. Wrapper: challenges/day-003-page-cache.sh.

Day 4 — Pipes, redirection, and backpressure

Day 4 uses a pipe as a bounded kernel buffer. `dup` of the write end is one buffer, not two. `dup2` onto standard output is how a child aims `write` at that buffer. Closing one writer leaves the pipe open; closing the last writer makes `read` return 0. Filling the pipe makes a nonblocking writer get `EAGAIN`, which is backpressure. A write of at most `PIPE_BUF` bytes is atomic: the empty pipe accepts all of it, and the full pipe returns `EAGAIN` rather than a prefix. Wrapper: challenges/day-004-pipes.sh.

Day 5 — select, poll, epoll, and readiness

Day 5 is the scheduling half of I/O. `select` and `poll` report an empty pipe and then a pipe that has bytes. Level-triggered `epoll` reports again while bytes remain after a partial read. Edge-triggered `epoll` does not repeat that edge; the lab drains until `EAGAIN`, then sees the next write as a new edge. A ready descriptor can still return a short read. Every wait in the lab uses a zero timeout so a missed event fails instead of sleeping. Wrapper: challenges/day-005-readiness.sh.

Day 6 — mmap, shared mappings, and data movement

Day 6 maps a file `MAP_SHARED`. A store is visible to `pread` before `msync`, because the mapping and the read share page-cache pages. `msync(MS_SYNC)` is then the same kind of request as `fsync`: completion is not a crash-durability proof. A later `pwrite` is visible in the mapping. The program also times one `pread` of the file and one copy from the mapping, prints both, and marks `performance_claim=none`. Wrapper: challenges/day-006-mmap.sh.

Day 7 — I/O debugging and a comparison report

Day 7 reads `/proc/self/fd` for a descriptor this process opened, then causes three failures on purpose: `read` after `close` (`EBADF`), `write` on a pipe with no reader and `SIGPIPE` ignored (`EPIPE`), and `epoll_ctl` on a closed fd (`EBADF`) plus a regular file (`EPERM`). Optional `strace` records the syscalls or prints `SKIP` when ptrace is unavailable. ToyOS at the Week 8/9 milestone has descriptors and a small pipe, not this VFS or page cache. Quote xv6 `file.c` or `pipe.c` only after pinning a revision. Wrapper: challenges/day-007-io-debug-report.sh.

Safety and evidence

The programs create temporary files only under `mkdtemp` directories in `/tmp`, unlink them, and remove the directories. The suite checks that those directories are gone. Children are waited for; a failing parent kills and reaps a child it still owns. Pipe fills set a 64 KiB capacity and stop. Readiness waits use a zero timeout. The `EINTR` parent arms a four-second alarm. Nothing in the lab opens a socket or asks for root.

`fsync` and `msync` success, and a readback that matches before either call, are page-cache and request results. The markers `durability_claim=none` and `msync_is_not_crash_proof=yes` are part of the test, not a comment you may drop. The `mmap` timings are one machine and one size.

Completion checkpoint

Finishing the wrappers does not complete mastery. Answer from the source and the logs, and leave a question blank when the run did not speak to it:

1. Which integer is the descriptor, which state lives in the open-file description, and which object is the file?
2. Which return is a short count, which is EOF, which is `EAGAIN`, and which is `EINTR`?
3. What did the second `open` see before `fsync`, and why is that not a crash-durability result?
4. Why did one closed writer fail to produce EOF, and why did the full-pipe `PIPE_BUF` write return `EAGAIN` instead of a prefix?
5. After a partial read, which of level-triggered and edge-triggered epoll reported again, and what did the edge-triggered path drain to?
6. Which bytes were visible through `MAP_SHARED` before `msync`, and which timing comparison are you refusing to generalize?
7. Which errno came from a closed descriptor, a pipe with no reader, and an `epoll_ctl` on a regular file?
8. Which kernel names did you re-read in tree `c9b2385d6793`, and which host `uname` actually ran the lab?

Do not auto-fill mastery, confidence, hours, or completion. The quizzes sit beside this overview and do not grade the report.

Day 1 — A Descriptor Is a Table Index

Target time: approximately 3 hours

- Why the kernel returns an integer: 30 minutes
- The chain from `files_struct` to `f_pos`: 45 minutes
- Sharing, close, and the two kinds of flags: 50 minutes
- One trace of open, dup, read, lseek, close: 30 minutes
- Mastery, then the wrapper: 25 minutes

> `open` returned `3`. What object does that integer name, and why does `lseek` on a duplicate move the next `read` on the original?

Week 8 crossed into the kernel with a saved user register frame and a checked copy. Week 9's ToyOS table already returns an integer for a slot that holds an offset and points at an inode. Week 10 hung `files` off `task_struct`. Week 11 showed `fork` duplicating that table and a pthread sharing it. Linux gives each of those layers its own lifetime. The integer is an index. The open file is a kernel object. The offset lives on the open file, so two indexes can share one position and two opens of one path can have two positions.

The excerpts below are from the local tree `5.10.134-15_rc1-10333-gc9b2385d6793` (commit `c9b2385d6793`). That tree is 5.10-based. Field layout in a later mainline release can move. The names quoted here are what this revision actually contains.

1. Why an integer is the handle

A user program needs to name an open file across many calls. The name it is allowed to hold is a small integer in its own memory. On the next syscall that integer arrives in an argument register, and the kernel uses it as a subscript into a table only the kernel allocates. Week 8's `copy_from_user` is the same rule for addresses: a value in a user register is an address in that process's page tables.

`files_lookup_fd_raw` in `include/linux/fdtable.h` refuses an index at or past `max_fds`, then clamps it with `array_index_nospec` before the load. `pick_file` in `fs/file.c` does the same clamp before it clears a slot. A page-table walk has the same shape. The tables decide the translation. The task's file table decides which open file the integer names.

ToyOS already uses the integer this way. In `projects/toyos/src/vfs.c`, `vfs_open` returns `VFS_FILE_BASE + i` for a free slot in `files[4]`. `file_for` rejects a number outside that range and a slot whose `open` flag is clear. The user program never holds `&files[i]`.

2. The table is an object the task points at

`task_struct` has a `files` pointer. Week 10's picture stopped there. The object on the other end of that pointer is `files_struct`, and the array inside it is an `fdtable`. Both live in `include/linux/fdtable.h`.

```
/* include/linux/fdtable.h :: struct fdtable :: tree c9b2385d6793
 * lines 27–44
 */
struct fdtable {
    unsigned int max_fds;
    struct file __rcu **fd;      /* current fd array */
    unsigned long *close_on_exec;
    unsigned long *open_fds;
    unsigned long *full_fds_bits;
    struct rcu_head rcu;
};

static inline bool close_on_exec(unsigned int fd, const struct fdtable *fdt)
{
    return test_bit(fd, fdt->close_on_exec);
}

static inline bool fd_is_open(unsigned int fd, const struct fdtable *fdt)
{
    return test_bit(fd, fdt->open_fds);
}
```

```
/* include/linux/fdtable.h :: struct files_struct :: tree c9b2385d6793
 * lines 49–68
 */
struct files_struct {
  /*
   * read mostly part
   */
    atomic_t count;
    bool resize_in_progress;
    wait_queue_head_t resize_wait;

    struct fdtable __rcu *fdt;
    struct fdtable fdtab;
  /*
   * written part on a separate cache line in SMP
   */
    spinlock_t file_lock ____cacheline_aligned_in_smp;
    unsigned int next_fd;
    unsigned long close_on_exec_init[1];
    unsigned long open_fds_init[1];
    unsigned long full_fds_bits_init[1];
    struct file __rcu * fd_array[NR_OPEN_DEFAULT];
};
```

`count` is how many tasks point at this table. Week 11's `copy_files` increments it under `CLONE_FILES`, so those threads share one array. `fdt` is the live `fdtable`. `fdtab` and `fd_array` are the embedded first table, `NR_OPEN_DEFAULT` slots. `file_lock` covers the written half. `fd[i]` is the open-file pointer. `open_fds` and `close_on_exec` are bits of the index.

```
task A                          task B (CLONE_FILES)
  files ──┐                      files ──┘
          v
     files_struct.count >= 2
       fdt -> fd[3] ──> struct file (one f_pos, one f_count)
              close_on_exec bit 3

task C after fork (CLONE_FILES clear)
  files -> a new files_struct
             fd[3] ──> the same struct file
             its own close_on_exec bit 3
```

`dup_fd` is the fork copy. The next section is the object it points at.

3. `struct file` is the open instance

The pointer in `fd[i]` aims at `struct file` in `include/linux/fs.h`. This is the object Week 9 called an open-file slot, with a kernel lifetime of its own.

```
/* include/linux/fs.h :: struct file :: tree c9b2385d6793
 * lines 959–978
 */
struct file {
    union {
        struct llist_node   fu_llist;
        struct rcu_head     fu_rcuhead;
    } f_u;
    struct path     f_path;
    struct inode        *f_inode;   /* cached value */
    const struct file_operations    *f_op;

    /*
     * Protects f_ep_links, f_flags.
     * Must not be taken from IRQ context.
     */
    spinlock_t      f_lock;
    enum rw_hint        f_write_hint;
    atomic_long_t       f_count;
    unsigned int        f_flags;
    fmode_t         f_mode;
    struct mutex        f_pos_lock;
    loff_t          f_pos;
```

`f_count` is the reference count. `get_file` (`include/linux/fs.h` lines 1008–1012) increments it. Each index that holds the pointer is one reference, and so is a kernel lookup that has not put the file yet. `f_pos` is the byte offset. `f_pos_lock` guards it when more than one reference exists. `f_flags` holds status such as `O_NONBLOCK`. `f_lock` guards that word. `f_mode` holds kernel bits such as `FMODE_READ` and `FMODE_LSEEK`. `f_op` is the method table. `f_inode` is the cached inode. `f_path` is the `include/linux/path.h` pair of mount and dentry; the dentry's `d_inode` is "where the name belongs."

`__alloc_file` in `fs/file_table.c` allocates with `kmem_cache_zalloc` and sets `f_count` to 1, so a new open file's `f_pos` starts at 0. Line 995 of `struct file`, `f_mapping`, points at the inode's `address_space`. Day 3 follows it. `do_dentry_open` stores `f_inode`, copies `inode->i_mapping`

into `f_mapping`, installs `f_op`, sets `FMODE_ATOMIC_POS` for a regular file or a directory, and clears `O_CREAT`, `O_EXCL`, `O_NOCTTY`, and `O_TRUNC` from `f_flags`. Those four bits were instructions for open.

4. Open reserves an index, then publishes the pointer

The user-visible `open` on this tree is a thin wrapper. `sys_open` may set `O_LARGEFILE`, then calls `do_sys_open(AT_FDCWD, ...)`. `do_sys_open` packs the flags into an `open_how` and calls `do_sys_openat2`.

```
/* fs/open.c :: do_sys_openat2, do_sys_open :: tree c9b2385d6793
 * lines 1218–1251
 */
static long do_sys_openat2(int dfd, const char __user *filename,
                struct open_how *how)
{
    struct open_flags op;
    int fd = build_open_flags(how, &op);
    struct filename *tmp;

    if (fd)
        return fd;

    tmp = getname(filename);
    if (IS_ERR(tmp))
        return PTR_ERR(tmp);

    fd = get_unused_fd_flags(how->flags);
    if (fd >= 0) {
        struct file *f = do_filp_open(dfd, tmp, &op);
        if (IS_ERR(f)) {
            put_unused_fd(fd);
            fd = PTR_ERR(f);
        } else {
            fsnotify_open(f);
            fd_install(fd, f);
        }
    }
    putname(tmp);
    return fd;
}

long do_sys_open(int dfd, const char __user *filename, int flags, umode_t mode)
{
    struct open_how how = build_open_how(flags, mode);
    return do_sys_openat2(dfd, filename, &how);
}
```

`getname` copies the user path before any slot exists. The pointer in the user register is a virtual address in the caller's page tables. `get_unused_fd_flags` then reserves an integer. `do_filp_open` builds the `struct file` only after that reservation. Failure calls `put_unused_fd`. Success calls `fd_install`. The `open_fds` bit stays set while `do_filp_open` runs, so two opens are not handed one index. `__put_unused_fd` (lines 586–591) clears the bit and may rewind `next_fd`, which is how a closed integer gets reused.

```
/* fs/file.c :: alloc_fd (tail), get_unused_fd_flags :: tree c9b2385d6793
 * lines 560–583
 */
    if (start <= files->next_fd)
        files->next_fd = fd + 1;

    __set_open_fd(fd, fdt);
    if (flags & O_CLOEXEC)
        __set_close_on_exec(fd, fdt);
    else
        __clear_close_on_exec(fd, fdt);
    error = fd;

out:
    spin_unlock(&files->file_lock);
    return error;
}

int __get_unused_fd_flags(unsigned flags, unsigned long nofile)
{
    return alloc_fd(0, nofile, flags);
}

int get_unused_fd_flags(unsigned flags)
{
    return __get_unused_fd_flags(flags, rlimit(RLIMIT_NOFILE));
}
```

`alloc_fd` holds `files->file_lock` until the bitmaps are updated. The ceiling is `RLIMIT_NOFILE`. `O_CLOEXEC` is still in this flags word; `build_open_flags` strips it from the open itself, and `alloc_fd` is what records the bit in `close_on_exec`.

```
/* fs/file.c :: fd_install :: tree c9b2385d6793
 * lines 620–642
 */
void fd_install(unsigned int fd, struct file *file)
{
    struct files_struct *files = current->files;
    struct fdtable *fdt;

    rcu_read_lock_sched();

    if (unlikely(files->resize_in_progress)) {
        rcu_read_unlock_sched();
        spin_lock(&files->file_lock);
        fdt = files_fdtable(files);
        WARN_ON(fdt->fd[fd] != NULL);
        rcu_assign_pointer(fdt->fd[fd], file);
        spin_unlock(&files->file_lock);
        return;
    }
    /* coupled with smp_wmb() in expand_fdtable() */
    smp_rmb();
    fdt = rcu_dereference_sched(files->fdt);
    BUG_ON(fdt->fd[fd] != NULL);
    rcu_assign_pointer(fdt->fd[fd], file);
    rcu_read_unlock_sched();
}
```

`rcu_assign_pointer` publishes the pointer. The comment at lines 616–617 says this consumes the file reference, as if the caller had called `fput`. The `BUG_ON` is the reservation's promise: the slot is empty. The comment above the function names a `dup2` that would install a different file between the bitmap update and this store.

5. `dup` and `fork` share `f_pos`; a second `open` does not

`dup` looks up the existing pointer, takes another reference, and installs that same pointer at a new index. `fork` does the same per slot inside `dup_fd` (`fs/file.c` lines 383–397): `get_file` on each live pointer, then `rcu_assign_pointer` into the child's new table.

```
/* fs/file.c :: sys_dup :: tree c9b2385d6793
 * lines 1228–1241
 */
SYSCALL_DEFINE1(dup, unsigned int, fildes)
{
    int ret = -EBADF;
    struct file *file = fget_raw(fildes);

    if (file) {
        ret = get_unused_fd_flags(0);
        if (ret >= 0)
            fd_install(ret, file);
        else
            fput(file);
    }
    return ret;
}
```

`fget_raw` is `__fget(fd, 0, 1)`, so it takes one reference and `fd_install` consumes it. Both indexes hold the same address and the same `f_pos`. `f_dupfd` (lines 1243–1254) is the `F_DUPFD` form: it calls `get_file` and then `fd_install`. `sys_dup` passes flags `0`, so `alloc_fd` clears close-on-exec on the new index. `F_DUPFD_CLOEXEC` passes `O_CLOEXEC` and sets the bit on the new index only.

The child after `fork` has private indexes and shares each `struct file`. Week 11's `copy_files` is the other switch: `CLONE_FILES` increments `files_struct.count` and shares the table. A second `open` calls `do_filp_open` again and gets a new `struct file` whose `f_pos` starts at 0.

`file_needs_f_pos_lock` (`fs/file.c` lines 997–1000) is true when `FMODE_ATOMIC_POS` is set and `file_count` is greater than one, or the inode is a directory. `__fdget_pos` then takes `f_pos_lock` around the update of that one `loff_t`.

6. Close clears one index and drops one reference

`sys_close` calls `close_fd`, which calls `__close_fd`. The slot is cleared before the file's release work runs.

```
/* fs/file.c :: pick_file, __close_fd :: tree c9b2385d6793
 * lines 646–679
 */
static struct file *pick_file(struct files_struct *files, unsigned fd)
{
    struct file *file = NULL;
    struct fdtable *fdt;

    spin_lock(&files->file_lock);
    fdt = files_fdtable(files);
    if (fd >= fdt->max_fds)
        goto out_unlock;
    fd = array_index_nospec(fd, fdt->max_fds);
    file = fdt->fd[fd];
    if (!file)
        goto out_unlock;
    rcu_assign_pointer(fdt->fd[fd], NULL);
    __put_unused_fd(files, fd);

out_unlock:
    spin_unlock(&files->file_lock);
    return file;
}

/*
 * The same warnings as for __alloc_fd()/__fd_install() apply here...
 */
int __close_fd(struct files_struct *files, unsigned fd)
{
    struct file *file;

    file = pick_file(files, fd);
    if (!file)
        return -EBADF;

    return filp_close(file, files);
}
```

`pick_file` clamps the index, unpublishes `fd[fd]`, and returns `NULL` for an empty slot, which `__close_fd` turns into `-EBADF`. `__put_unused_fd` clears `open_fds` and may rewind `next_fd`, so the next `alloc_fd` can reuse the integer. After that the old `int` is a stale name: a later `open` may install a different `struct file` at the same index. The bits in the user's variable survived; the object did not.

`filp_close` (`fs/open.c` lines 1332–1349) calls `f_op->flush` when present, drops POSIX locks for this owner, then `fput`. `fput_many` (`fs/file_table.c` lines 335–354) subtracts from `f_count` with `atomic_long_sub_and_test` and queues `__fput` only when the count hits zero. A `dup`'d file stays alive when one index closes. `__fput` calls `f_op->release` and drops the path. The inode has its own `i_count`.

`sys_close` has a second rule, written on the syscall itself: if `close_fd` returns a restart code, `sys_close` rewrites it to `-EINTR`. The comment says the close cannot be restarted because the table entry was already cleared. A signal arriving during flush must not replay an operation that has already unpublished the slot.

7. `O_CLOEXEC` is a slot bit; status flags live on the file

`build_open_flags` (`fs/open.c` lines 1042–1057) sets `strip = FMODE_NONOTIFY | O_CLOEXEC` and clears those bits from the local flags that describe the open. The comment says `O_CLOEXEC` is not relevant to `struct open_flags`. `how->flags` is unchanged, so `get_unused_fd_flags(how->flags)` still sees the bit and `alloc_fd` records it in `close_on_exec`. Two stores, two objects.

`fcntl` keeps the split visible to user space. `do_fcntl` in `fs/fcntl.c`:

```
/* fs/fcntl.c :: do_fcntl :: tree c9b2385d6793
 * lines 337–356
 */
    switch (cmd) {
    case F_DUPFD:
        err = f_dupfd(arg, filp, 0);
        break;
    case F_DUPFD_CLOEXEC:
        err = f_dupfd(arg, filp, O_CLOEXEC);
        break;
    case F_GETFD:
        err = get_close_on_exec(fd) ? FD_CLOEXEC : 0;
        break;
    case F_SETFD:
        err = 0;
        set_close_on_exec(fd, arg & FD_CLOEXEC);
        break;
    case F_GETFL:
        err = filp->f_flags;
        break;
    case F_SETFL:
        err = setfl(fd, filp, arg);
        break;
```

`F_GETFD` / `F_SETFD` read and write the bitmap for this index. `F_GETFL` returns `filp->f_flags`. `F_SETFL` calls `setfl`, which holds `filp->f_lock` and writes

```
filp->f_flags = (arg & SETFL_MASK) | (filp->f_flags & ~SETFL_MASK);
```

with `SETFL_MASK` equal to `O_APPEND | O_NONBLOCK | O_NDELAY | O_DIRECT | O_NOATIME` (`fs/fcntl.c` line 33, assignment at line 80). `dup` shares those status bits, because `dup` shares the `struct file`. `F_SETFD` changes one index's bitmap bit. `do_close_on_exec` walks that bitmap at `exec` and closes those indexes. Another index, or another task's table, can still hold the `struct file`.

8. `lseek` writes the shared `f_pos`

`read` and `write` also move `f_pos`. Day 2 follows that path. The seek path is the one that writes the field on purpose.

```
/* fs/read_write.c :: vfs_llseek, ksys_lseek :: tree c9b2385d6793
 * lines 291–320
 */
loff_t vfs_llseek(struct file *file, loff_t offset, int whence)
{
    loff_t (*fn)(struct file *, loff_t, int);

    fn = no_llseek;
    if (file->f_mode & FMODE_LSEEK) {
        if (file->f_op->llseek)
            fn = file->f_op->llseek;
    }
    return fn(file, offset, whence);
}
EXPORT_SYMBOL(vfs_llseek);

static off_t ksys_lseek(unsigned int fd, off_t offset, unsigned int whence)
{
    off_t retval;
    struct fd f = fdget_pos(fd);
    if (!f.file)
        return -EBADF;

    retval = -EINVAL;
    if (whence <= SEEK_MAX) {
        loff_t res = vfs_llseek(f.file, offset, whence);
        retval = res;
        if (res != (loff_t)retval)
            retval = -EOVERFLOW;    /* LFS: should only happen on 32 bit platforms */
    }
    fdput_pos(f);
    return retval;
}
```

`fdget_pos` is `__fdget_pos`: the index becomes a `struct file *`, and the position mutex is taken when `file_needs_f_pos_lock` says so. A missing file is `-EBADF`. `vfs_llseek` calls `no_llseek`, which returns `-ESPIPE`, unless `FMODE_LSEEK` is set and `f_op->llseek` is present. `default_llseek` writes `file->f_pos` under `inode_lock`. The method writes the shared field, so `lseek` on one duplicate moves the position `read` will use on the other. `fdput_pos` drops the lookup reference.

9. Execution trace: open, dup, read, lseek, close

One task, already running, with its own `files_struct`. The path names a regular file. Numbers here are a walk through the calls, not a trace of one PID.

```
user                          kernel, this task
open(path, O_RDONLY)          sys_open
  path pointer in rdi           getname: copy the user path
  flags in rsi                  get_unused_fd_flags → index 3
  mode in rdx                   do_filp_open → new struct file
                                f_pos = 0, f_count = 1
                                fd_install(3, file)
                                return 3 in rax

dup(3)                        sys_dup
  3 in rdi                      fget_raw(3): f_count = 2
                                get_unused_fd_flags(0) → index 4
                                close_on_exec bit 4 cleared
                                fd_install(4, same file)
                                return 4

read(3, buf, 1)               ksys_read
                                fdget_pos(3): same struct file
                                copy f_pos (0) to a local
                                one byte copied, local pos = 1
                                f_pos = 1

read(4, buf, 1)               ksys_read
                                fd[4] is the same struct file
                                starts at f_pos 1

lseek(4, 0, SEEK_SET)         ksys_lseek → vfs_llseek
                                f_op->llseek writes f_pos = 0
                                a later read on fd 3 starts at 0

close(4)                      pick_file clears fd[4]
                                filp_close → fput
                                f_count = 1, file still alive

close(3)                      pick_file clears fd[3]
                                fput → count reaches 0
                                __fput → f_op->release
```

A second `open` of the same path allocates another `struct file` at `f_pos == 0`. Day 2 expands the `read` steps. fd 3 and fd 4 are two slots and one `f_pos`.

10. A small C picture of one shared offset

This is the user-space side of the trace. `open` and `dup` are libc wrappers around the syscalls above. The checks make the shared position visible: the second `read` must observe the byte after the first.

```
#include <fcntl.h>
#include <unistd.h>

int main(void)
{
    char byte;
    int fd = open("offset-demo", O_RDWR | O_CREAT | O_TRUNC, 0600);
    int alias;
    if (fd < 0)
        return 1;
    if (write(fd, "abcd", 4) != 4)
        return 1;
    if (lseek(fd, 0, SEEK_SET) != 0)
        return 1;
    alias = dup(fd);
    if (alias < 0)
        return 1;
    if (read(fd, &byte, 1) != 1 || byte != 'a')
        return 1;
    if (read(alias, &byte, 1) != 1 || byte != 'b')
        return 1;
    if (lseek(alias, 0, SEEK_SET) != 0)
        return 1;
    if (read(fd, &byte, 1) != 1 || byte != 'a')
        return 1;
    close(alias);
    close(fd);
    return 0;
}
```

`alias` and `fd` are two `int`s on this stack. After `dup` they select one `struct file`: the second `read` returns `'b'`, and `lseek` on `alias` rewinds `read` on `fd`. A second `open` would be a new `f_pos`. After `close(fd)` the variable still holds the integer, and `next_fd` may already have reused the slot.

11. Registers, locks, and the ToyOS table

On x86-64, `arch/x86/entry/entry_64.S` records the `syscall` registers: `rax` is the call number, `rdi` / `rsi` / `rdx` are the first three arguments, and the return comes back in `rax`. `arch/x86/entry/syscalls/syscall_64.tbl` numbers `read` 0, `write` 1, `open` 2, `close` 3, `lseek` 8, `dup` 32, `fcntl` 72. A negative `rax` is an errno. AArch64 takes the number from `regs->regs[8]` in `do_el0_svc` and the arguments from `orig_x0` plus `regs[1]` through `regs[5]` (`arch/arm64/include/asm/syscall.h`). The generic table uses different numbers: `openat` 56, `close` 57, `read` 63, `write` 64 (`include/uapi/asm-generic/unistd.h`). RISC-V holds the number in `a7` and stores the return to `PT_A0` (`arch/riscv/kernel/entry.S`). The C behind the stub is `do_sys_open` and `ksys_lseek` on each of them.

`getname` translates the path with the caller's `mm->pgd`. The new table and `struct file` are kernel objects. `files->file_lock` covers allocation and `pick_file`. `f_pos_lock` covers `f_pos` once `file_count` exceeds one. `f_lock` covers `f_flags`, so `F_SETFL` of `O_NONBLOCK` is visible on every index that shares the file. Threads with `CLONE_FILES` share the table.

ToyOS (projects/toyos/src/vfs.c) keeps one `files[4]`. Each slot has its own `offset`, set to 0 by `vfs_open`, and `file_for` rejects a bad index. The bytes live in the inode array. Linux adds `f_count`, the shared `f_pos`, the per-index close-on-exec bit, and a table tasks can share or duplicate. Day 2 follows `f_op`. Day 3 follows `f_mapping`.

12. Mastery: Explain · Draw · Observe · Build

Run `../challenges/day-001-descriptors.sh` after this explanation. The wrapper is the evidence step.

Explain

A descriptor is an index in the calling task's `fdtable`. The table hangs off `files_struct`, which hangs off `task_struct.files`. `fd[i]` is a `struct file *`. That object holds `f_count`, `f_pos`, `f_flags`, `f_mode`, `f_op`, `f_inode`, and `f_path`. `open` reserves the index in `get_unused_fd_flags` before `do_filp_open`, then `fd_install` publishes the pointer and consumes one reference. `dup` and `fork`'s `dup_fd` install another reference to the same `struct file`, so `f_pos` is shared. A second `open` builds another `struct file`. `close` clears one slot and `fput`s one reference. `O_CLOEXEC` is a `close_on_exec` bit on the index. `O_NONBLOCK` and the other `SETFL_MASK` bits live in `f_flags`.

Draw

1. One `files_struct`, its `fdtable`, two indexes pointing at one `struct file`, and that file's `f_pos` and `f_inode`.
2. The open sequence: `getname`, reserve index, `do_filp_open`, `fd_install`, with the `put_unused_fd` arrow on the error path.
3. After `fork`: two `files_struct`s, two indexes, one `struct file`. Beside it, two threads with `CLONE_FILES`: one `files_struct`.

Observe

On the lab run, record each integer the program prints and say which kernel object it indexes. Mark which indexes share one `f_pos`, which close drops a reference without freeing the file, and which close drops the last reference. Record an `EBADF` from a stale or empty index as the `pick_file` miss.

Build

Extend the C picture in section 10 only by a second `open` of the same path. Predict, before you run it, that the new index has its own `f_pos` at 0 while `alias` still sees the shared position. Then name the call in `do_sys_openat2` that allocates that second `struct file`.

Why? notebook

1. Why is a descriptor an integer in a user register, rather than a kernel pointer the user could pass back?
2. Why does `get_unused_fd_flags` run before `do_filp_open`?
3. Why does `fd_install` consume a reference?
4. Why does `sys_dup` pass `0` to `get_unused_fd_flags`?
5. Why can `lseek` on one duplicate move `read` on the other?

6. Why is `O_CLOEXEC` stripped in `build_open_flags` and still honored by `alloc_fd`?
7. Why can `close` return `-EINTR` without restarting?
8. Why does `__fdget_pos` lock `f_pos` when `file_count` is greater than one?
9. Why does ToyOS store `offset` in `struct file` and the bytes in `struct inode`?

Next: Day 2 follows `read` and `write` from that index through a short transfer, a zero end, and a negative errno.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Descriptors

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 12-linux-io-and-readiness/challenges/day-001-descriptors.sh
```

- View lab source on GitHub

Day 2 — A Count Is a Request, a Return Is Progress

Target time: approximately 3 hours

- Why the kernel may move fewer bytes than you asked: 30 minutes
- One `read` and one `write` through the VFS: 50 minutes
- Sleep, `EAGAIN`, and `EINTR`: 40 minutes
- The copy across the user boundary, then a loop: 35 minutes
- Mastery, then the wrapper: 25 minutes

> You asked `read` for 4096 bytes and it returned 200. Has the call failed, finished the file, or merely paused?

Day 1 made a descriptor an index into a `struct file`, and `f_pos` the position that index shares with its duplicates. This lesson is what a later `read` or `write` of that index is allowed to do with a user buffer. The number you pass is a ceiling. The number that comes back is how far this attempt got.

The excerpts are from the local tree `5.10.134-15_rc1-10333-gc9b2385d6793` (commit `c9b2385d6793`), the same 5.10-based tree as Day 1.

1. Why the call is allowed to stop early

A disk file, a pipe, a socket, and a device do not have the same notion of "the next bytes." A regular file has an `i_size` and a page cache. A pipe has a ring of buffers and a count of writers. A socket has a receive queue that the peer fills when it chooses. A device method decides how many bytes its hardware has ready. One kernel function that promised to fill the user buffer would have to sleep, retry, and hide those differences.

`vfs_read` and `vfs_write` therefore perform one attempt. They check the open file, check the user range, and call `f_op->read` or `f_op->write` (or the `*_iter` method) once. Whatever `ssize_t` that method returns is what the syscall returns. The VFS does not loop until the caller's count is satisfied. That loop is the caller's job, because only the caller knows whether a short result should be retried, reported, or treated as "try again when the descriptor is ready."

ToyOS is the small version of a short read. `vfs_read` sets `count = length < available ? length : available`, copies that many bytes from the inode array, advances `offset`, and returns `count`. At the end of the inode that count is 0. `vfs_write` refuses a write that does not fit, returning `-28`, so the toy does not yet have a short write. Linux uses one signed return for all three outcomes, on every kind of object that supplies `file_operations`.

2. Three results, and the libc translation

Inside the kernel the return is an `ssize_t`:

- a positive value is the number of bytes moved;

- `0` is a completed attempt that moved nothing, which for a nonzero request at end-of-file, or on a pipe whose last writer is gone, is the end;
- a negative value is `-errno`.

`SYSCALL_DEFINE3(read)` returns `ksys_read` directly, so the register that comes back to user mode holds that `ssize_t`. Libc's `read` and `write` are a second translation: a negative kernel return becomes `-1` and `errno` is set to the positive code. A program that treats any return below the requested count as failure will retry a finished file forever, or abandon a transfer that had already moved bytes.

Two zeros must stay apart. The header comment in `fs/pipe.c` lines 68–69 says a read with `count = 0` should return 0. `generic_file_buffered_read` in `mm/filemap.c` does the same when `iov_iter_count(iter)` is 0 (lines 2611–2612). That zero is "you asked for nothing." End-of-file is a zero returned for a nonzero request once `iocb->ki_pos >= isize` (lines 2651–2652) and no bytes were copied. The caller's loop distinguishes them by the count it passed in.

`include/uapi/asm-generic/errno.h` line 22 defines `EWOULDBLOCK` as `EAGAIN`. On this tree they are one code. A test for either name matches the same return.

3. `ksys_read` looks up the index and moves `f_pos` only on progress

```
/* fs/read_write.c :: file_ppos, ksys_read :: tree c9b2385d6793
 * lines 617–640
 */
/* file_ppos returns &file->f_pos or NULL if file is stream */
static inline loff_t *file_ppos(struct file *file)
{
    return file->f_mode & FMODE_STREAM ? NULL : &file->f_pos;
}

ssize_t ksys_read(unsigned int fd, char __user *buf, size_t count)
{
    struct fd f = fdget_pos(fd);
    ssize_t ret = -EBADF;

    if (f.file) {
        loff_t pos, *ppos = file_ppos(f.file);
        if (ppos) {
            pos = *ppos;
            ppos = &pos;
        }
        ret = vfs_read(f.file, buf, count, ppos);
        if (ret >= 0 && ppos)
            f.file->f_pos = pos;
        fdput_pos(f);
    }
    return ret;
}
```

`fdget_pos` is Day 1's lookup: a bad index stays `-EBADF`, and `f_pos_lock` is taken when the file has `FMODE_ATOMIC_POS` and more than one reference. `file_ppos` returns `NULL` for `FMODE_STREAM`, so a stream has no shared offset to update. Otherwise the current `f_pos` is copied to a local `pos`, and `vfs_read` receives a pointer to that local. The store back to `f.file-`

`>f_pos` happens only when `ret >= 0`. An error leaves the shared offset where it was. A short success moves it by however many bytes the method claimed, because the method updates `*ppos` and this function publishes that local back.

`fdput_pos` drops the lookup reference and unlocks the position mutex if it was taken. The user's `buf` is still a `__user` pointer. Nothing in `ksys_read` dereferences it.

4. `vfs_read` checks, caps, and calls the method once

```
/* fs/read_write.c :: vfs_read :: tree c9b2385d6793
 * lines 476–505
 */
ssize_t vfs_read(struct file *file, char __user *buf, size_t count, loff_t *pos)
{
    ssize_t ret;

    if (!(file->f_mode & FMODE_READ))
        return -EBADF;
    if (!(file->f_mode & FMODE_CAN_READ))
        return -EINVAL;
    if (unlikely(!access_ok(buf, count)))
        return -EFAULT;

    ret = rw_verify_area(READ, file, pos, count);
    if (ret)
        return ret;
    if (count > MAX_RW_COUNT)
        count =  MAX_RW_COUNT;

    if (file->f_op->read)
        ret = file->f_op->read(file, buf, count, pos);
    else if (file->f_op->read_iter)
        ret = new_sync_read(file, buf, count, pos);
    else
        ret = -EINVAL;
    if (ret > 0) {
        fsnotify_access(file);
        add_rchar(current, ret);
    }
    inc_syscr(current);
    return ret;
}
```

`FMODE_READ` and `FMODE_CAN_READ` are the open file's own capability, set when the file was opened. A write-only description fails here, before any byte moves. `access_ok(buf, count)` asks whether the whole range is a user address this kernel is willing to copy. Failure is `-EFAULT`, and `f_pos` will not move because `ksys_read` stores it only for `ret >= 0`.

`MAX_RW_COUNT` is `(INT_MAX & PAGE_MASK)` in `include/linux/fs.h` line 2542. A request larger than that is shortened before the method runs, so a regular file can return a short count while `i_size` still has more bytes. That short count is progress. The next call continues at the updated `f_pos`.

The method pointer is the object Day 1's `f_op` selected. A pipe, a socket, and a device node each install their own `read` or `read_iter`. `vfs_read` calls one of them once and returns. `new_sync_read` builds a one-element `iov_iter` and calls `read_iter`; it too is one attempt.

For a regular file that attempt often lands in `generic_file_read_iter`. Its comment in `mm/filemap.c` lines 2719–2737 says the routine is for filesystems that use the page cache directly, that `IOCB_NOWAIT` means `-EAGAIN` when the read would wait, and that the return is the number of bytes copied, including partial reads. `generic_file_buffered_read` returns `written ? written : error` (line 2714). Bytes already copied win over a later fault: the positive count is what the caller must account for, and the offset has already moved.

5. `rw_verify_area` rejects a nonsense range

```
/* fs/read_write.c :: rw_verify_area :: tree c9b2385d6793
 * lines 371–401
 */
    inode = file_inode(file);
    if (unlikely((ssize_t) count < 0))
        return retval;

    /*
     * ranged mandatory locking does not apply to streams - it makes sense
     * only for files where position has a meaning.
     */
    if (ppos) {
        loff_t pos = *ppos;

        if (unlikely(pos < 0)) {
            if (!unsigned_offsets(file))
                return retval;
            if (count >= -pos) /* both values are in 0..LLONG_MAX */
                return -EOVERFLOW;
        } else if (unlikely((loff_t) (pos + count) < 0)) {
            if (!unsigned_offsets(file))
                return retval;
        }

        if (unlikely(inode->i_flctx && mandatory_lock(inode))) {
            retval = locks_mandatory_area(inode, file, pos, pos + count - 1,
                    read_write == READ ? F_RDLCK : F_WRLCK);
            if (retval < 0)
                return retval;
        }
    }

    return security_file_permission(file,
                read_write == READ ? MAY_READ : MAY_WRITE);
```

`retval` was set to `-EINVAL` at the top of the function (line 369). A `count` that does not fit in `ssize_t` is rejected, which is why a huge request cannot look like a negative length later. A negative position is `-EINVAL` unless the file uses unsigned offsets, and a position plus count that wraps is the same class of rejection, or `-EOVERFLOW` when the unsigned case still cannot represent the range. Streams pass a `NULL` position from `file_ppos`, so the position checks are skipped, and the comment says mandatory locking applies only where a position means something. The last call is the LSM hook. None of this copies bytes. It decides whether the one attempt may start.

6. `vfs_write` and `ksys_write` are the same contract in the other direction

```
/* fs/read_write.c :: vfs_write :: tree c9b2385d6793
 * lines 585–615
 */
ssize_t vfs_write(struct file *file, const char __user *buf, size_t count, loff_t *pos)
{
    ssize_t ret;

    if (!(file->f_mode & FMODE_WRITE))
        return -EBADF;
    if (!(file->f_mode & FMODE_CAN_WRITE))
        return -EINVAL;
    if (unlikely(!access_ok(buf, count)))
        return -EFAULT;

    ret = rw_verify_area(WRITE, file, pos, count);
    if (ret)
        return ret;
    if (count > MAX_RW_COUNT)
        count =  MAX_RW_COUNT;
    file_start_write(file);
    if (file->f_op->write)
        ret = file->f_op->write(file, buf, count, pos);
    else if (file->f_op->write_iter)
        ret = new_sync_write(file, buf, count, pos);
    else
        ret = -EINVAL;
    if (ret > 0) {
        fsnotify_modify(file);
        add_wchar(current, ret);
    }
    inc_syscw(current);
    file_end_write(file);
    return ret;
}
```

```
/* fs/read_write.c :: ksys_write :: tree c9b2385d6793
 * lines 647–665
 */
ssize_t ksys_write(unsigned int fd, const char __user *buf, size_t count)
{
    struct fd f = fdget_pos(fd);
    ssize_t ret = -EBADF;

    if (f.file) {
        loff_t pos, *ppos = file_ppos(f.file);
        if (ppos) {
            pos = *ppos;
            ppos = &pos;
        }
        ret = vfs_write(f.file, buf, count, ppos);
        if (ret >= 0 && ppos)
            f.file->f_pos = pos;
        fdput_pos(f);
    }

    return ret;
}
```

The mode bits, `access_ok`, `rw_verify_area`, and the `MAX_RW_COUNT` cap match `vfs_read`. `file_start_write` / `file_end_write` bracket the method. The method is again one call. A positive return is bytes taken from the user buffer. `ksys_write` publishes `f_pos` only when `ret >= 0`. A short write is progress: the next call must pass the tail, not the original buffer. Day 3 follows a buffered write after this return and shows why a positive count is still not a durable store.

`O_NONBLOCK` and `O_APPEND` live in `f_flags`, the status word Day 1 put on the `struct file`. `dup` shares them. A nonblocking duplicate and the original index wait, or refuse to wait, together.

7. A pipe sleeps, returns `EAGAIN`, or returns what it has

`pipe_read` in `fs/pipe.c` is one method `vfs_read` can land on. After it has copied whatever is already buffered, lines 340–347 decide among end, short progress, and waiting. No writers breaks the loop: `ret` stays 0 if nothing was copied, which is the end, or stays at the short count if something was. A nonzero `ret` breaks immediately, so a blocking pipe read is still short when some data was already there. Only the empty, still-open case continues. `O_NONBLOCK` in `f_flags` sets `-EAGAIN` and does not sleep.

The sleep is the rest of that path:

```
/* fs/pipe.c :: pipe_read :: tree c9b2385d6793
 * lines 344–378
 */
        if (filp->f_flags & O_NONBLOCK) {
            ret = -EAGAIN;
            break;
        }
        __pipe_unlock(pipe);

        /*
         * We only get here if we didn't actually read anything.
         *
         * However, we could have seen (and removed) a zero-sized
         * pipe buffer, and might have made space in the buffers
         * that way.
         *
         * You can't make zero-sized pipe buffers by doing an empty
         * write (not even in packet mode), but they can happen if
         * the writer gets an EFAULT when trying to fill a buffer
         * that already got allocated and inserted in the buffer
         * array.
         *
         * So we still need to wake up any pending writers in the
         * _very_ unlikely case that the pipe was full, but we got
         * no data.
         */
        if (unlikely(was_full))
            wake_up_interruptible_sync_poll(&pipe->wr_wait, EPOLLOUT | EPOLLWRNORM);
        kill_fasync(&pipe->fasync_writers, SIGIO, POLL_OUT);

        /*
         * But because we didn't read anything, at this point we can
         * just return directly with -ERESTARTSYS if we're interrupted,
         * since we've done any required wakeups and there's no need
         * to mark anything accessed. And we've dropped the lock.
         */
        if (wait_event_interruptible_exclusive(pipe->rd_wait, pipe_readable(pipe)) < 0)
            return -ERESTARTSYS;
```

`wait_event_interruptible_exclusive` is the task leaving the run queue until `pipe_readable` is true or a signal is pending. That is the scheduler from Week 7 applied to an I/O wait queue: the core is free to run someone else, and this task's registers sit in its saved frame. The comment says the wait returns `-ERESTARTSYS` when interrupted, and that this path has not copied bytes, so there is no partial progress to preserve.

`pipe_write` (lines 547–556) is the mirror. A full pipe with `O_NONBLOCK` sets `-EAGAIN` when `ret` is still 0. `signal_pending` sets `-ERESTARTSYS` on the same "nothing committed yet" rule.

The user-visible `EINTR` is what the signal path does with `-ERESTARTSYS`. On x86, `handle_signal` in `arch/x86/kernel/signal.c`:

```
/* arch/x86/kernel/signal.c :: handle_signal :: tree c9b2385d6793
 * lines 793–813
 */
    /* Are we from a system call? */
    if (syscall_get_nr(current, regs) >= 0) {
        /* If so, check system call restarting.. */
        switch (syscall_get_error(current, regs)) {
        case -ERESTART_RESTARTBLOCK:
        case -ERESTARTNOHAND:
            regs->ax = -EINTR;
            break;

        case -ERESTARTSYS:
            if (!(ksig->ka.sa.sa_flags & SA_RESTART)) {
                regs->ax = -EINTR;
                break;
            }
            fallthrough;
        case -ERESTARTNOINTR:
            regs->ax = regs->orig_ax;
            regs->ip -= 2;
            break;
        }
    }
```

If the handler lacks `SA_RESTART`, `rax` becomes `-EINTR` and libc's `read` returns `-1` with `errno == EINTR`. If `SA_RESTART` is set, the original syscall number is restored and `ip` moves back over the two-byte `syscall` instruction, so the instruction runs again. A loop that retries `EINTR` is correct for the first case. The second case never surfaces in user space. Either way the program must not treat `EINTR` as a permanent failure of the descriptor.

`generic_perform_write` has a direct `-EINTR` as well, when `fatal_signal_pending(current)` is true (`mm/filemap.c` lines 3572–3574). That one is not `-ERESTARTSYS`. It is already the errno a write attempt returns if it has not copied anything yet.

8. The bytes cross a checked boundary

`access_ok` in `vfs_read` is the first gate. The copy itself is the second. Week 8's `copy_from_user` walked the user page tables one byte at a time because a user register holds an address in that process's map. This tree's helper is the same rule with a wider copy.

```
/* include/linux/uaccess.h :: _copy_from_user :: tree c9b2385d6793
 * lines 152–164
 */
static inline __must_check unsigned long
_copy_from_user(void *to, const void __user *from, unsigned long n)
{
    unsigned long res = n;
    might_fault();
    if (!should_fail_usercopy() && likely(access_ok(from, n))) {
        instrument_copy_from_user(to, from, n);
        res = raw_copy_from_user(to, from, n);
    }
    if (unlikely(res))
        memset(to + (n - res), 0, res);
    return res;
}
```

```
/* include/linux/uaccess.h :: copy_from_user, copy_to_user :: tree c9b2385d6793
 * lines 188–202
 */
static __always_inline unsigned long __must_check
copy_from_user(void *to, const void __user *from, unsigned long n)
{
    if (likely(check_copy_size(to, n, false)))
        n = _copy_from_user(to, from, n);
    return n;
}

static __always_inline unsigned long __must_check
copy_to_user(void __user *to, const void *from, unsigned long n)
{
    if (likely(check_copy_size(from, n, true)))
        n = _copy_to_user(to, from, n);
    return n;
}
```

The return is the number of bytes not copied. Zero means the whole range moved. `might_fault` is the permission for this path to take a page fault: the user virtual address is translated through the calling task's page tables, the same walk Week 5 and Week 7 built. A hole in that map, or a permission the user is not allowed, stops the copy. The comment at line 89 says only `copy_from_user` zero-fills the uncopied tail of the kernel destination. The kernel does not leave leftover stack bytes from a short copy visible as if they had been read from the user.

`__user` on the pointer is the type-level reminder. A C pointer in kernel code and a user virtual address are different objects. Dereferencing the user pointer with an ordinary load skips `access_ok`, the fault-in, and the zero-fill. That is the bug Week 8's checked copy exists to prevent.

A read's copy is usually `copy_to_user` or `copy_page_to_iter`: kernel bytes into the user map. A write's copy is the other way. In `generic_perform_write`, `iov_iter_fault_in_readable` brings the user page in before `write_begin`, and a short `copied` count is progress or a fault, not a silent success. The VFS return is still one `ssize_t`.

```
user virtual address in rsi / x1 / a1
        |
        |  page-table walk of current->mm
        v
access_ok, then raw_copy_*
        |
        v
kernel buffer or page-cache page
        |
        v
ssize_t in the return register
        positive  -> caller advances its cursor by that many
        0         -> end, for a nonzero request
        negative  -> errno, f_pos unchanged when ksys_* saw ret < 0
```

9. A loop that means what it says

This is the caller side of the contract. `read` here is libc: `-1` and `errno`, not the kernel's negative `ssize_t`. The cursor `got` is the only record of progress. `EINTR` retries the same tail. `EAGAIN` returns the bytes already taken so the caller can wait for readiness, or `-1` when nothing has been taken yet.

```
#include <errno.h>
#include <unistd.h>

ssize_t read_full(int fd, void *buf, size_t count)
{
    unsigned char *p = buf;
    size_t got = 0;

    while (got < count) {
        ssize_t n = read(fd, p + got, count - got);
        if (n > 0) {
            got += (size_t)n;
            continue;
        }
        if (n == 0)
            return (ssize_t)got;
        if (errno == EINTR)
            continue;
        if (errno == EAGAIN)
            return got ? (ssize_t)got : -1;
        return -1;
    }
    return (ssize_t)got;
}
```

A `write` loop has the same shape, with one extra care: a return of 0 from `write` is not end-of-file. It is a stall the caller has to define a policy for. The kernel side of a pipe write returns `-EAGAIN` or sleeps; it does not use 0 as "the reader went away" in the snippet above. A read's 0 is the end. Mixing those two zeros is how a copy loop spins.

The `fd` in this function is Day 1's index. If another thread shares the `files_struct` and closes it, the next `read` can return `EBADF`. If a `dup` or a `fork` shares the `struct file`, this loop and the other task advance one `f_pos` under `f_pos_lock`. The cursor `got` is private to this call. `f_pos` is not.

10. Execution trace: a short read, a sleep, a signal

One task, descriptor 3 a blocking pipe, descriptor 4 a regular file positioned at the start of a 100-byte file. The user asks for 4096.

```
read(4, buf, 4096)                  ksys_read
  rdi = 4, rsi = buf, rdx = 4096      fdget_pos, local pos = f_pos
                                      vfs_read: access_ok ok
                                      read_iter -> page cache
                                      100 bytes copied, pos = 100
                                      f_pos = 100, return 100
  libc returns 100                    loop adds 100 and calls again

read(4, buf+100, 3996)                ki_pos >= i_size, written == 0
                                      return 0
  libc returns 0                      loop treats 0 as end
                                      total 100, not an error

read(3, buf, 4096)  pipe empty,       pipe_read: ret == 0, writers exist,
O_NONBLOCK clear                      O_NONBLOCK clear
                                      wait_event_interruptible_exclusive
                                      task sleeps on pipe->rd_wait

writer deposits 50 bytes              wake, copy 50, ret = 50
                                      if (ret) break
  libc returns 50                     short, and the pipe still has a writer

signal during the wait,               pipe_read returns -ERESTARTSYS
no bytes copied, SA_RESTART clear     handle_signal sets regs->ax = -EINTR
  libc returns -1, errno = EINTR      loop continues, same buffer
```

The regular-file pair is a full transfer made of a short count and a zero. The pipe pair is a sleep, then a short count, because data was available and the method returned it. The signal case copies nothing, so retrying does not skip a byte and does not double-count one.

11. Architecture lens

Day 1 named the registers. For `read` and `write` they hold the descriptor, the user address, and the count. On x86-64, `arch/x86/entry/entry_64.S` lines 76–85: `rax` is the syscall number (0 for `read`, 1 for `write` in `syscall_64.tbl`), `rdi` is arg0, `rsi` arg1, `rdx` arg2. The return `ssize_t` comes back in `rax`. AArch64 uses `regs->regs[8]` as the number and `orig_x0` plus `regs[1]`..`regs[5]` as arguments, so a read's descriptor, buffer, and count are `x0`, `x1`, `x2`. RISC-V keeps the number in `a7` and returns in `a0`. The generic numbers differ (`read` is 63, `write` is 64) and the C is `ksys_read` / `ksys_write` either way.

The page-table root stays the caller's `mm->pgd` for the whole copy. `might_fault` and `raw_copy_from_user` are loads and stores whose addresses are translated by that root. A fault in the kernel's copy path is the same class of event as a user load that misses: the tables either name a frame the access may use, or they do not. The kernel then turns the failure into the uncopied count, and `vfs_read` turns a failed `access_ok` into `-EFAULT` before the method runs.

`f_pos_lock` is the mutex from Day 1. Two threads, or a parent and child sharing the open file, do not each run this update unsynchronized. `files->file_lock` is not held across the wait: `pipe_read` drops the pipe lock before `wait_event_interruptible_exclusive`, which is why a closer in another thread can proceed. The sleeper rechecks the pipe when it wakes.

12. Mastery: Explain · Draw · Observe · Build

Run `../challenges/day-002-stream-io.sh` after this explanation.

Explain

`vfs_read` and `vfs_write` check mode, `access_ok`, and `rw_verify_area`, cap the count at `MAX_RW_COUNT`, and call one method. A positive `ssize_t` is progress and is the amount `ksys_read` / `ksys_write` add to `f_pos`. Zero for a nonzero request is the end. A negative value is `-errno`, and `f_pos` stays put. `EAGAIN` is the nonblocking pipe's refusal to sleep. `-ERESTARTSYS` from `pipe_read` becomes `-EINTR` or a restarted `syscall`, depending on `SA_RESTART`. The user pointer is copied through `access_ok` and `copy_from_user` / `copy_to_user`, which walk the caller's page tables.

Draw

1. The path from the user count through `ksys_read`, `vfs_read`, and one `f_op` call, with three arrows out: positive, zero, negative.
2. A pipe's decision: no writers, already copied something, `O_NONBLOCK`, or `wait_event_interruptible_exclusive`.
3. The user buffer as a virtual address, the page-table walk, and the kernel buffer on the other side of `copy_to_user`.

Observe

On the lab run, record a short positive return, a zero, an `EINTR` retry, and an `EAGAIN` from a nonblocking descriptor. For each, say whether `f_pos` moved and which source condition produced it: `MAX_RW_COUNT` or end-of-file, `pipe_read`'s `if (ret) break`, `O_NONBLOCK`, or `handle_signal`.

Build

Extend `read_full` with a `write_full` that advances a cursor by each positive return, retries `EINTR`, and on `EAGAIN` returns the bytes already written. Predict the `f_pos` value after a 100-byte file is read with a 4096-byte request that returns 100 and then 0.

Why? notebook

1. Why does `vfs_read` call the method once?
2. Why does `ksys_read` assign `f_pos` only when `ret >= 0`?
3. Why can a regular file return a short count while bytes remain?
4. Why is a zero from `read` of a nonzero count the end, while a zero request is also a zero return?
5. Why does `pipe_read` return `-EAGAIN` only when `ret` is still 0?
6. Why does `-ERESTARTSYS` sometimes appear in user space as `EINTR`?
7. Why does `_copy_from_user` zero the uncopied tail?
8. Why is the user buffer address translated by `mm->pgd` during the copy?
9. Why does ToyOS `vfs_read` return a short count, while its `vfs_write` returns `-28` when the bytes do not fit?

Next: Day 3 follows a positive `write` into a dirty page-cache page, and shows what `fsync` waits for.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Stream Io

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 12-linux-io-and-readiness/challenges/day-002-stream-io.sh
```

- View lab source on GitHub

Day 3 — Dirty Memory Is Not a Durable Write

Target time: approximately 3 hours

- Why a path, an open file, and a cached page are different objects: 30 minutes
- A buffered write dirties a page: 45 minutes
- Writeback, `fsync`, and the error that comes back later: 45 minutes
- Holes, a trace, and the ToyOS boundary: 35 minutes
- Mastery, then the wrapper: 25 minutes

> `write` returned 4096. The machine loses power before `fsync` returns. Which of those bytes can a later `open` be expected to see?

Day 1 separated the descriptor, the `struct file`, and the inode. Day 2 separated a requested count from the bytes one attempt actually moved. A positive `write` on a regular file still has a third gap. The bytes can sit in RAM, tagged dirty, while the storage underneath has not taken them. `fsync` is the call that waits. A crash, or a writeback error, lives in that gap.

The excerpts are from the local tree `5.10.134-15_rc1-10333-gc9b2385d6793` (commit `c9b2385d6793`). This revision's `mm/filemap.c` still speaks of `struct page`, `PageDirty`, and the page cache. Later mainline trees renamed much of that to folios. The names below are this tree's names.

1. Why the cache is not the open file

A path is input to lookup. A dentry remembers a name; `d_inode` is "where the name belongs" (`include/linux/dcache.h`). An inode is the object: mode, size, and the operations. An open file is one description of that object, with its own `f_pos` and `f_count`. None of those is a cache of the bytes.

The cache hangs off the inode. `inode_init_always` in `fs/inode.c` sets `mapping` to `&inode->i_data` (line 137), points `mapping->host` back at the inode (line 182), and stores that pointer in `inode->i_mapping` (line 195). `do_dentry_open` copies it onto the open file (`fs/open.c` line 796, `f->f_mapping = inode->i_mapping`). Two opens of one inode share one `address_space`. They do not share `f_pos`. Day 1's duplicates share `f_pos` and also share this cache, because they share the file and therefore the inode pointer inside it.

```
/* include/linux/fs.h :: struct address_space :: tree c9b2385d6793
 * lines 470–501
 */
struct address_space {
    struct inode        *host;
    struct xarray       i_pages;
    gfp_t           gfp_mask;
    atomic_t        i_mmap_writable;
#ifdef CONFIG_READ_ONLY_THP_FOR_FS
    /* number of thp, only for non-shmem files */
    atomic_t        nr_thps;
#endif
#ifdef CONFIG_DUPTEXT
    /* number of dup text, in pages */
    atomic_t                nr_duptext;
#endif
    struct rb_root_cached   i_mmap;
    struct rw_semaphore i_mmap_rwsem;
    unsigned long       nrpages;
    unsigned long       nrexceptional;
    pgoff_t         writeback_index;
    const struct address_space_operations *a_ops;
    unsigned long       flags;
    errseq_t        wb_err;
    spinlock_t      private_lock;
    struct list_head    private_list;
    void            *private_data;

    struct fast_reflink_work *fast_reflink_work;

    CK_KABI_RESERVE(1)
    CK_KABI_RESERVE(2)
    CK_KABI_RESERVE(3)
    CK_KABI_RESERVE(4)
} __attribute__((aligned(sizeof(long)))) __randomize_layout;
```

`i_pages` is the cache, an xarray of pages. `nrpages` counts them. `a_ops` is how this inode reads a missing page, begins a write, and writes pages back. `wb_err` is an `errseq_t` for the last writeback error. `host` is the inode. Just below the struct, lines 508–511 name the xarray tags: `PAGECACHE_TAG_DIRTY`, `PAGECACHE_TAG_WRITEBACK`, and `PAGECACHE_TAG_TOWRITE`. Dirty and writeback are marks on the cache, not bits in the user's buffer.

`i_mmap` is the tree of VM areas that map this file. A buffered `read` copies from a cache page into a user buffer translated by the process page tables. The page frame can later be the same frame a PTE names. The process page tables and `i_pages` are two indexes.

```
path string                 lookup input
    |
    v
dentry.d_inode  ---------> inode
                            i_mapping -------+
                            i_data           |
                            i_size           v
                                        address_space
                                          i_pages[index] -> struct page
                                          PAGECACHE_TAG_DIRTY
                                          PAGECACHE_TAG_WRITEBACK
                                          wb_err
struct file
  f_pos          one open's cursor
  f_mapping  ---> the inode's address_space
  f_op       ---> read / write / fsync
```

The page index is a file offset in page units. `generic_perform_write` computes the byte offset inside a page as `pos & (PAGE_SIZE - 1)`. `grab_cache_page_write_begin` takes that cache position as a `pgoff_t index`. The process's page tables are not consulted to find it. They are consulted to find the user buffer the bytes are copied from.

2. The method table is where read, write, and fsync meet

```
/* include/linux/fs.h :: struct file_operations :: tree c9b2385d6793
 * lines 1981–2001
 */
struct file_operations {
    struct module *owner;
    fop_flags_t fop_flags;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
    ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
    ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
    int (*iopoll)(struct kiocb *kiocb, struct io_comp_batch *,
            unsigned int flags);
    int (*iterate) (struct file *, struct dir_context *);
    int (*iterate_shared) (struct file *, struct dir_context *);
    __poll_t (*poll) (struct file *, struct poll_table_struct *);
    long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
    long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    unsigned long mmap_supported_flags;
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *, fl_owner_t id);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, loff_t, loff_t, int datasync);
```

Day 2's `vfs_read` ends by calling one of those read methods:

```
/* fs/read_write.c :: vfs_read :: tree c9b2385d6793
 * lines 493–505
 */
    if (file->f_op->read)
        ret = file->f_op->read(file, buf, count, pos);
    else if (file->f_op->read_iter)
        ret = new_sync_read(file, buf, count, pos);
    else
        ret = -EINVAL;
    if (ret > 0) {
        fsnotify_access(file);
        add_rchar(current, ret);
    }
    inc_syscr(current);
    return ret;
}
```

`new_sync_read` calls `read_iter`. For a page-cache filesystem that method is `generic_file_read_iter` (`mm/filemap.c` lines 2719–2724). A miss finds or allocates a `struct page` in `i_pages` and copies it out. `page_cache_sync_ra` (`mm/readahead.c` lines 581–607) widens the fill through `ondemand_readahead`, unless random access or congestion forces a smaller read. `f_op->fsync` is a different method: write accepts bytes, and `fsync` pushes them and reports failure.

3. A buffered write copies into a page and marks it dirty

`grab_cache_page_write_begin` is the page-cache vocabulary of this tree: a `struct page`, a page-cache index, a locked page.

```
/* mm/filemap.c :: grab_cache_page_write_begin :: tree c9b2385d6793
 * lines 3510–3528
 */
/*
 * Find or create a page at the given pagecache position. Return the locked
 * page. This function is specifically for buffered writes.
 */
struct page *grab_cache_page_write_begin(struct address_space *mapping,
                    pgoff_t index, unsigned flags)
{
    struct page *page;
    int fgp_flags = FGP_LOCK|FGP_WRITE|FGP_CREAT;

    if (flags & AOP_FLAG_NOFS)
        fgp_flags |= FGP_NOFS;
    if (flags & AOP_FLAG_DONTCACHE)
        fgp_flags |= FGP_DONTCACHE;

    page = pagecache_get_page(mapping, index, fgp_flags,
            mapping_gfp_mask(mapping));
    if (page)
        wait_for_stable_page(page);
```

`generic_perform_write` is the loop a buffered `write_iter` uses. Each pass covers one page, faults the user side in first, and copies.

```
/* mm/filemap.c :: generic_perform_write :: tree c9b2385d6793
 * lines 3556–3588
 */
        offset = (pos & (PAGE_SIZE - 1));
        bytes = min_t(unsigned long, PAGE_SIZE - offset,
                        iov_iter_count(i));

again:
        /*
         * Bring in the user page that we will copy from _first_.
         * Otherwise there's a nasty deadlock on copying from the
         * same page as we're writing to, without it being marked
         * up-to-date.
         */
        if (unlikely(iov_iter_fault_in_readable(i, bytes))) {
            status = -EFAULT;
            break;
        }

        if (fatal_signal_pending(current)) {
            status = -EINTR;
            break;
        }

        if (iocb->ki_flags & IOCB_DONTCACHE)
            flags |= AOP_FLAG_DONTCACHE;

        status = a_ops->write_begin(file, mapping, pos, bytes, flags,
                        &page, &fsdata);
        if (unlikely(status < 0))
            break;

        if (mapping_writably_mapped(mapping))
            flush_dcache_page(page);

        copied = iov_iter_copy_from_user_atomic(page, i, offset, bytes);
```

The user pointer is still a virtual address. `iov_iter_fault_in_readable` walks the caller's page tables and brings the source page in before the filesystem locks the cache page. That order is the deadlock comment: the source and the destination might be the same frame if the process has the file mapped. `fatal_signal_pending` produces `-EINTR` directly, the case Day 2 distinguished from a pipe's `-ERESTARTSYS`. `write_begin` is the filesystem method that finds or creates the cache page. The copy then moves bytes from the user map into that `struct page`.

The function returns `written ? written : status` (line 3617). A positive count is how far the copy got. `ksys_write` will publish that into `f_pos`. The page is not yet a completed device write.

For a buffer-head filesystem, `write_end` reaches `__block_commit_write`, which marks the touched buffers dirty:

```
/* fs/buffer.c :: __block_commit_write :: tree c9b2385d6793
 * lines 2077–2101
 */
    do {
        block_end = block_start + blocksize;
        if (block_end <= from || block_start >= to) {
            if (!buffer_uptodate(bh))
                partial = 1;
        } else {
            set_buffer_uptodate(bh);
            mark_buffer_dirty(bh);
        }
        if (buffer_new(bh))
            clear_buffer_new(bh);

        block_start = block_end;
        bh = bh->b_this_page;
    } while (bh != head);

    /*
     * If this is a partial write which happened to make all buffers
     * uptodate then we can optimize away a bogus readpage() for
     * the next read(). Here we 'discover' whether the page went
     * uptodate as a result of this (potentially partial) write.
     */
    if (!partial)
        SetPageUptodate(page);
    return 0;
```

`mark_buffer_dirty` says what "dirty" means here:

```
/* fs/buffer.c :: mark_buffer_dirty :: tree c9b2385d6793
 * lines 1099–1111
 */
/**
 * mark_buffer_dirty - mark a buffer_head as needing writeout
 * @bh: the buffer_head to mark dirty
 *
 * mark_buffer_dirty() will set the dirty bit against the buffer, then set
 * its backing page dirty, then tag the page as dirty in the page cache
 * and then attach the address_space's inode to its superblock's dirty
 * inode list.
 *
 * mark_buffer_dirty() is atomic.  It takes bh->b_page->mapping->private_lock,
 * i_pages lock and mapping->host->i_lock.
 */
void mark_buffer_dirty(struct buffer_head *bh)
```

The tag is `PAGECACHE_TAG_DIRTY` on `i_pages`. The page's `PageDirty` bit is set along the way (`TestSetPageDirty` in the body at line 1134). The inode joins the superblock's dirty list. All of that is bookkeeping in memory. `generic_write_end` may also extend `i_size` while the page is still locked (the comment at lines 2183–2185), then `mark_inode_dirty` if the size changed. A larger `i_size` is metadata. It is not a promise that the device has the new bytes.

`balance_dirty_pages_ratelimited` can start cleaning so the cache does not grow without bound. That cleaning is writeback. The `write` syscall can return while tagged pages remain.

4. Writeback moves dirty pages; `fsync` waits

```
/* mm/filemap.c :: __filemap_fdatawrite_range :: tree c9b2385d6793
 * lines 414–433
 */
int __filemap_fdatawrite_range(struct address_space *mapping, loff_t start,
                loff_t end, int sync_mode)
{
    int ret;
    struct writeback_control wbc = {
        .sync_mode = sync_mode,
        .nr_to_write = LONG_MAX,
        .range_start = start,
        .range_end = end,
    };

    if (!mapping_can_writeback(mapping) ||
        !mapping_tagged(mapping, PAGECACHE_TAG_DIRTY))
        return 0;

    wbc_attach_fdatawrite_inode(&wbc, mapping->host);
    ret = do_writepages(mapping, &wbc);
    wbc_detach_inode(&wbc);
    return ret;
}
```

The comment above this function (lines 404–410) calls `WB_SYNC_ALL` a data-integrity operation: a dirty page must be waited on, not skipped. `do_writepages` is the handoff from the cache to `a_ops`. Below that call are the filesystem's block mapping and the block device. This lesson stops at that handoff. The page-cache fact is the tag. Starting writeback moves a page from the dirty tag toward `PAGECACHE_TAG_WRITEBACK`.

Waiting is a second step:

```
/* mm/filemap.c :: filemap_fdatawait_range :: tree c9b2385d6793
 * lines 561–582
 */
/**
 * filemap_fdatawait_range - wait for writeback to complete
 * @mapping:        address space structure to wait for
 * @start_byte:     offset in bytes where the range starts
 * @end_byte:       offset in bytes where the range ends (inclusive)
 *
 * Walk the list of under-writeback pages of the given address space
 * in the given range and wait for all of them.  Check error status of
 * the address space and return it.
 *
 * Since the error status of the address space is cleared by this function,
 * callers are responsible for checking the return value and handling and/or
 * reporting the error.
 *
 * Return: error status of the address space.
 */
int filemap_fdatawait_range(struct address_space *mapping, loff_t start_byte,
                loff_t end_byte)
{
    __filemap_fdatawait_range(mapping, start_byte, end_byte);
    return filemap_check_errors(mapping);
}
```

`__filemap_fdatawait_range` looks up `PAGECACHE_TAG_WRITEBACK` and calls `wait_on_page_writeback` on each `struct page`. `filemap_check_errors` (lines 373–384) returns `-ENOSPC` if `AS_ENOSPC` is set and `-EIO` if `AS_EIO` is set, clearing the bit it reports. The comment says the caller must check that return. A wait that ignores it has observed the sleep and dropped the error.

The syscall entry does not call these helpers by name. It calls the filesystem method:

```
/* fs/sync.c :: do_fsync, fsync, fdatasync :: tree c9b2385d6793
 * lines 220–240
 */
static int do_fsync(unsigned int fd, int datasync)
{
    struct fd f = fdget(fd);
    int ret = -EBADF;

    if (f.file) {
        ret = vfs_fsync(f.file, datasync);
        fdput(f);
    }
    return ret;
}

SYSCALL_DEFINE1(fsync, unsigned int, fd)
{
    return do_fsync(fd, 0);
}

SYSCALL_DEFINE1(fdatasync, unsigned int, fd)
{
    return do_fsync(fd, 1);
}
```

`vfs_fsync` calls `vfs_fsync_range(file, 0, LLONG_MAX, datasync)`. That helper's comment (lines 184–192) says it writes back data in the range and metadata for the file, and that `datasync` limits metadata to what is needed to access the modified data. The code then requires `f_op->fsync` and calls it. `fsync` passes `datasync = 0`. `fdatasync` passes 1. On x86-64 those syscalls are numbers 74 and 75 in `syscall_64.tbl`. The generic table numbers them 82 and 83. The descriptor is still Day 1's index. `fdget` turns it into the `struct file` whose `f_mapping` names the cache.

5. The failure is recorded on the mapping

A writeback error is stored where a later `fsync` can find it.

```
/* include/linux/pagemap.h :: mapping_set_error :: tree c9b2385d6793
 * lines 39–70
 */
/**
 * mapping_set_error - record a writeback error in the address_space
 * @mapping: the mapping in which an error should be set
 * @error: the error to set in the mapping
 *
 * When writeback fails in some way, we must record that error so that
 * userspace can be informed when fsync and the like are called.  We endeavor
 * to report errors on any file that was open at the time of the error.  Some
 * internal callers also need to know when writeback errors have occurred.
 *
 * When a writeback error occurs, most filesystems will want to call
 * mapping_set_error to record the error in the mapping so that it can be
 * reported when the application calls fsync(2).
 */
static inline void mapping_set_error(struct address_space *mapping, int error)
{
    if (likely(!error))
        return;

    /* Record in wb_err for checkers using errseq_t based tracking */
    __filemap_set_wb_err(mapping, error);

    /* Record it in superblock */
    if (mapping->host)
        errseq_set(&mapping->host->i_sb->s_wb_err, error);

    /* Record it in flags for now, for legacy callers */
    if (error == -ENOSPC)
        set_bit(AS_ENOSPC, &mapping->flags);
    else
        set_bit(AS_EIO, &mapping->flags);
}
```

Three records: `mapping->wb_err`, the superblock's `s_wb_err`, and the legacy `AS_ENOSPC` / `AS_EIO` bits that `filemap_check_errors` reads. `mark_buffer_write_io_error` in `fs/buffer.c` (lines 1146–1161) calls `mapping_set_error` with `-EIO` when a buffer write fails. The `write` that dirtied the page may already have returned a positive count. The error belongs to a later integrity call.

`__delete_from_page_cache`'s helper states the loss plainly. The comment at `mm/filemap.c` lines 219–222 says a dirty page at deletion "signals a bug and loss of unwritten data." Dirty means the bytes are still the cache's responsibility. Dropping the page before writeback finishes drops the only copy.

That is the failure boundary. A positive `write` means the copy and the dirty tag happened. `fsync` returning 0 means `f_op->fsync` returned 0. `-EIO` or `-ENOSPC` means an error recorded on the mapping, or an error the method returned, reached the syscall. Between those two returns the dirty page is still RAM.

6. A hole is an extent fact, not a missing cache page

`generic_file_llseek_size` has no hole map of its own. Its `SEEK_DATA` arm (`fs/read_write.c` lines 111–118) treats every offset below the end as data. Its `SEEK_HOLE` arm (lines 119–127) says there is a virtual hole at the end of the file and, when the offset is below that end, returns `i_size`. A filesystem that stores holes replaces `llseek`. ext4 does:

```
/* fs/ext4/file.c :: ext4_llseek :: tree c9b2385d6793
 * lines 868–888
 */
loff_t ext4_llseek(struct file *file, loff_t offset, int whence)
{
    struct inode *inode = file->f_mapping->host;
    loff_t maxbytes = ext4_get_maxbytes(inode);

    switch (whence) {
    default:
        return generic_file_llseek_size(file, offset, whence,
                        maxbytes, i_size_read(inode));
    case SEEK_HOLE:
        inode_lock_shared(inode);
        offset = iomap_seek_hole(inode, offset,
                     &ext4_iomap_report_ops);
        inode_unlock_shared(inode);
        break;
    case SEEK_DATA:
        inode_lock_shared(inode);
        offset = iomap_seek_data(inode, offset,
                     &ext4_iomap_report_ops);
        inode_unlock_shared(inode);
        break;
```

The inode lock is shared. The answer comes from `iomap_seek_hole`. `fs/ext4/extents_status.c` lines 20–38 describe the extent-status tree that answers `SEEK_DATA` / `SEEK_HOLE`. A hole is a range of file offsets with no allocated blocks. `i_size`, that map, and `i_pages` are three records of the file.

7. Execution trace: write, dirty, writeback, fsync, error

One regular file, buffer-head `a_ops`, descriptor 3, `f_pos` at 0. The user calls `write(3, buf, 4096)` and the copy succeeds. Then `fsync(3)`.

```
write(3, buf, 4096)
  ksys_write -> vfs_write -> write_iter
    generic_perform_write
      offset = 0 within the page
      iov_iter_fault_in_readable(user buf)     page tables of current->mm
      a_ops->write_begin                       grab or create struct page
                                               at this page-cache index
      copy from user into that page
      __block_commit_write
        mark_buffer_dirty
          PageDirty, PAGECACHE_TAG_DIRTY
          inode on the superblock dirty list
      i_size updated if the write extended the file
    return 4096
  f_pos = 4096
  user sees 4096                               the device has not been asked

later, memory pressure or a flusher
  __filemap_fdatawrite_range
    mapping is tagged DIRTY
    do_writepages                 filesystem block mapping, then the device

fsync(3), ext4
  do_fsync(fd, 0) -> ext4_sync_file          fs/ext4/fsync.c
    file_write_and_wait_range                mm/filemap.c 824–839
      __filemap_fdatawrite_range(WB_SYNC_ALL)
      __filemap_fdatawait_range
      file_check_and_advance_wb_err
    journal commit, or ext4_fsync_nojournal
    blkdev_issue_flush if needs_barrier
    file_check_and_advance_wb_err once more
  return 0, or -EIO / -ENOSPC from that cursor

power loss after write returned and before fsync returned
  the dirty page was still RAM
```

The positive `write` and the successful `fsync` are two syscalls. Code that treats the first as durability has stopped at the dirty tag. Code that ignores `fsync`'s return has waited and dropped `filemap_check_errors`.

8. A small C picture of the two calls

```
#include <unistd.h>

int store(int fd, const void *buf, size_t count)
{
    const char *p = buf;
    size_t got = 0;

    while (got < count) {
        ssize_t n = write(fd, p + got, count - got);
        if (n < 0)
            return -1;
        if (n == 0)
            return -1;
        got += (size_t)n;
    }
    return fsync(fd);
}
```

`write` looping until `got == count` is Day 2: each positive return is progress in the page cache and in `f_pos`. `fsync` is the integrity call on the same descriptor. Its zero means the method returned zero. `fdatasync` is the same path with `datasync` set. Neither call is a substitute for checking the other's return. A short `write` that is not retried leaves a prefix dirty and the tail unsent. An `fsync` error after a successful `write` means the prefix was accepted into the cache and the integrity path later failed.

9. Registers, locks, and ToyOS

`fsync`'s descriptor arrives in the same argument register as `read`'s: `rdi` on x86-64, `x0` on AArch64, `a0` on RISC-V. The syscall number is in `rax`, `regs[8]`, or `a7`. The return is 0 or a negative errno. No user buffer is copied. The objects are reached from `current->files`.

`generic_write_end` updates `i_size` while the page is locked so writeback cannot zero past `i_size` (`fs/buffer.c` lines 2183–2185). `mark_buffer_dirty` takes `mapping->private_lock`, the `i_pages` lock, and `mapping->host->i_lock`. `ext4_llseek` takes `inode_lock_shared`. The user buffer is translated by the process page tables. The cache page is found through `i_pages`. When `mapping_writably_mapped` is set, the write flushes the data cache so a user mapping of that page sees the new bytes.

ToyOS writes `inode->data` before `vfs_write` returns. The offset lives on the open slot. Linux returns at the dirty tag and waits when the program calls `fsync`.

10. Mastery: Explain · Draw · Observe · Build

Run `../challenges/day-003-page-cache.sh` after this explanation.

Explain

The page cache is `inode->i_mapping`, an `address_space` whose `i_pages` hold `struct page` objects tagged dirty or writeback. `struct file` points at that mapping and keeps its own `f_pos`. A buffered `write` copies into a cache page, `mark_buffer_dirty` tags it, and `write` can return. `__filemap_fdatawrite_range` starts writeback through `do_writepages`. `fsync` and `fdatasync` are `do_fsync` with `datasync` 0 and 1, and they call `f_op->fsync`.

`mapping_set_error` records a writeback failure on `wb_err` and on `AS_EIO` / `AS_ENOSPC`. A hole, on ext4, is an extent-map answer from `iomap_seek_hole`. The generic seek has one virtual hole at `i_size`.

Draw

1. Path, dentry, inode, `address_space`, and two `struct file`s sharing `f_mapping` with different `f_pos` values.
2. One page: user buffer via the process page tables, cache page via `i_pages`, dirty tag, then writeback tag.
3. The timeline: `write` returns, writeback runs, `fsync` waits, and the arrow where `mapping_set_error` is reported.

Observe

On the lab run, record a `write` return and the later `fsync` return as two observations. Note whether a short `write` was retried before `fsync`. If the run reports a writeback error, name `mapping_set_error` and `filemap_check_errors` as the record and the report.

Build

In the C picture, split `store` so the caller can tell a `write` error from an `fsync` error. Predict `f_pos` and the dirty tag after `write` returns 4096 and before `fsync` is called.

Why? notebook

1. Why do two opens of one inode share `i_pages` and not `f_pos`?
2. Why can `write` return before `do_writepages` runs?
3. Why does `generic_perform_write` fault the user page in before `write_begin`?
4. Why is `PAGECACHE_TAG_DIRTY` a different mark from `PAGECACHE_TAG_WRITEBACK`?
5. Why does `filemap_fdatawait_range` return `filemap_check_errors`?
6. Why do `fsync` and `fdatasync` differ by the `datasync` argument?
7. Why does generic `SEEK_HOLE` return `i_size`, while ext4 calls `iomap_seek_hole`?
8. Why does ToyOS finish the byte update inside `vfs_write`?

Next: the descriptor, the short transfer, and the dirty page are the three objects later weeks use when they talk about readiness and durable storage.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Page Cache

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 12-linux-io-and-readiness/challenges/day-003-page-cache.sh
```

- View lab source on GitHub

Day 4 — Pipes, Redirection, and Backpressure

Target time: approximately 3 hours

> A pipe is a bounded queue of bytes in the kernel. Each end is a file the process already knows how to read and write. The bound is what forces a fast writer to sleep.

The excerpts in this lesson are Linux 5.10.134 at tree `c9b2385d6793`. Record `uname -r` before you treat a line number as true of the machine you are tracing. Names and widths move between releases. The POSIX contract for `pipe`, `read`, `write`, and `dup2` is the application interface. This file is one kernel's implementation of that contract.

ToyOS already has the smallest version of the idea. Week 8's pipe is one 64-byte array, two fixed descriptors, and a copy in each direction. Today's work is the part that array leaves out: two real file objects, a ring that reuses slots, sleep when the ring is full or empty, end-of-file when the last writer releases its file, and `SIGPIPE` when a write finds no reader.

1. Why the bytes cannot live in either process

A producer and a consumer are two address spaces. Week 10 gives each task its own page tables. A store the producer executes updates a physical page that the consumer's page tables do not name. Handing the consumer a raw pointer into the producer would be handing it an address that translates, in the consumer, to something else or to nothing.

The kernel is the place both tasks can enter. A `write` follows the syscall path from Week 8: registers cross in, the kernel copies out of the caller's buffer, and the call returns a count. A later `read` copies into a different user buffer. The queue between those copies is kernel memory.

The queue is bounded. An unbounded queue lets a fast producer allocate until the machine is out of pages. A bound turns "the consumer is behind" into a sleep, the same shape as the condition-variable wait in Week 11. The predicates are "there is a free slot" and "there is a byte, or no writer left." The lock is the pipe mutex. The queues are `rd_wait` and `wr_wait`.

```
producer address space          kernel                      consumer address space
  user buffer --write()-->   pipe ring of pages   --read()-->  user buffer
                             head produces
                             tail consumes
                             full  -> writer sleeps on wr_wait
                             empty -> reader sleeps on rd_wait
```

The user-visible object is still a file. Week 9 already made a descriptor an index into a per-task table of open files. A pipe does not add a new syscall family for data movement. It adds a pair of files whose `read_iter` and `write_iter` implement this queue.

2. One ring, two counters, pages for slots

`include/linux/pipe_fs_i.h` in this tree describes the object. The comment is the data model. The fields under it are what `pipe_read` and `pipe_write` actually update.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: include/linux/pipe_fs_i.h
 * lines: 34–64
 * function: struct pipe_inode_info (definition)
 */
/**
 *  struct pipe_inode_info - a linux kernel pipe
 *  @mutex: mutex protecting the whole thing
 *  @rd_wait: reader wait point in case of empty pipe
 *  @wr_wait: writer wait point in case of full pipe
 *  @head: The point of buffer production
 *  @tail: The point of buffer consumption
 *  @note_loss: The next read() should insert a data-lost message
 *  @max_usage: The maximum number of slots that may be used in the ring
 *  @ring_size: total number of buffers (should be a power of 2)
 *  @nr_accounted: The amount this pipe accounts for in user->pipe_bufs
 *  @tmp_page: cached released page
 *  @readers: number of current readers of this pipe
 *  @writers: number of current writers of this pipe
 *  @files: number of struct file referring this pipe (protected by ->i_lock)
 *  @r_counter: reader counter
 *  @w_counter: writer counter
 *  @poll_usage: is this pipe used for epoll, which has crazy wakeups?
 *  @fasync_readers: reader side fasync
 *  @fasync_writers: writer side fasync
 *  @bufs: the circular array of pipe buffers
 *  @user: the user who created this pipe
 *  @watch_queue: If this pipe is a watch_queue, this is the stuff for that
 **/
struct pipe_inode_info {
    struct mutex mutex;
    wait_queue_head_t rd_wait, wr_wait;
    unsigned int head;
    unsigned int tail;
    unsigned int max_usage;
    unsigned int ring_size;
```

`head` and `tail` are stored unmasked. The comment at lines 62–66 of the same header says they wrap naturally, and that the ring is a power of two, at most `2^31`. A slot is `bufs[index & (ring_size - 1)]`, a `struct pipe_buffer` with a `struct page *`, an offset, and a length. `PIPE_SIZE` is `PAGE_SIZE`. One slot is one page of pipe bytes, not one userspace `write`.

`alloc_pipe_info` starts from `PIPE_DEF_BUFFERS` (16). Over the soft per-user limit it shrinks to `PIPE_MIN_DEF_BUFFERS` (2, at the top of `fs/pipe.c`), because a one-slot pipe blocks a writer that still owes the pipe a page. `pipe_max_size` is 1048576, the comment's cap for a non-root `F_SETPIPE_SZ`. A 4096-byte page makes the default ring 16 pages. That product is not a second named constant.

`pipe_empty` is `head == tail`. `pipe_full` is `head - tail >= max_usage` at the `pipe_read` and `pipe_write` call sites. Unsigned subtraction is what lets the ring be full with no spare slot reserved to tell full from empty. `__pipe_lock` takes `pipe->mutex` directly. While it is held, `head`, `tail`, and the buffer lengths move together. The waiter is whichever task next enters `read` or `write`, the kernel analogue of the pthread mutex in Week 11's queue.

3. `pipe()` builds two files and publishes two descriptors

`get_pipe_inode` allocates the inode on the pipe pseudo-filesystem, calls `alloc_pipe_info`, and at lines 882–885 sets `inode->i_pipe`, `pipe->files = 2`, `pipe->readers = pipe->writers = 1`, and `inode->i_fop = &pipefifo_fops`.

`files` starts at 2 because `create_pipe_files` is about to build two `struct file` objects. `readers` and `writers` start at 1: one read end, one write end. Those counters are not the number of tasks and not the number of descriptor slots. `fork` and `dup` add references to an existing `struct file`. `pipe_release` runs later, from `__fput` in `fs/file_table.c`, when the last reference to that file is dropped. Until then `writers` stays 1 even if three processes hold the write end.

`create_pipe_files` is the split into directions. The first `struct file` is write-only. The read end is a clone of it. Both store the same `pipe_inode_info` in `private_data`.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/pipe.c
 * lines: 926–948
 * function: create_pipe_files
 */
    f = alloc_file_pseudo(inode, pipe_mnt, "",
                O_WRONLY | (flags & (O_NONBLOCK | O_DIRECT)),
                &pipefifo_fops);
    if (IS_ERR(f)) {
        free_pipe_info(inode->i_pipe);
        iput(inode);
        return PTR_ERR(f);
    }

    f->private_data = inode->i_pipe;

    res[0] = alloc_file_clone(f, O_RDONLY | (flags & O_NONBLOCK),
                  &pipefifo_fops);
    if (IS_ERR(res[0])) {
        put_pipe_info(inode, inode->i_pipe);
        fput(f);
        return PTR_ERR(res[0]);
    }
    res[0]->private_data = inode->i_pipe;
    res[1] = f;
    stream_open(inode, res[0]);
    stream_open(inode, res[1]);
    return 0;
```

`res[0]` is the read end. `res[1]` is the write end. `O_NONBLOCK` is copied onto both. `O_DIRECT` is applied to the write end only; `pipe_write` later treats that flag as packet mode, which section 9 keeps separate from the ordinary byte stream. `stream_open` marks both as streams, so they have no shared file position that a seek could name. `pipefifo_fops` wires `.read_iter` to `pipe_read`, `.write_iter` to `pipe_write`, `.poll` to `pipe_poll`, and `.release` to `pipe_release`.

`pipe2` and `pipe` both land in `do_pipe2`. `pipe` passes flags 0.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/pipe.c
 * lines: 1001–1021
 * function: do_pipe2
 */
static int do_pipe2(int __user *fildes, int flags)
{
    struct file *files[2];
    int fd[2];
    int error;

    error = __do_pipe_flags(fd, files, flags);
    if (!error) {
        if (unlikely(copy_to_user(fildes, fd, sizeof(fd)))) {
            fput(files[0]);
            fput(files[1]);
            put_unused_fd(fd[0]);
            put_unused_fd(fd[1]);
            error = -EFAULT;
        } else {
            fd_install(fd[0], files[0]);
            fd_install(fd[1], files[1]);
        }
    }
    return error;
}
```

`SYSCALL_DEFINE2(pipe2)` returns `do_pipe2`. `__do_pipe_flags` calls `create_pipe_files`, then `get_unused_fd_flags` twice. `copy_to_user` publishes the indexes, and only then does `fd_install` make them live. A faulting copy `fput`s both files, drops the reserved indexes, and returns `-EFAULT` with nothing installed. That is the Week 8 user-copy rule: do not publish a kernel object if the user pointer did not accept the name.

After a successful `pipe`, the process has two descriptors and one queue. A `write` on the read index fails the direction check before `pipe_write` runs, because that `struct file` was opened `O_RDONLY`. A `read` on the write index fails the same way. The direction lives in the file, not in a bit the user has to pass on every call.

4. `fork` plus `dup2` is how a shell aims the queue

A pipe inside one process is only a queue the process can use to talk to itself. A pipeline needs a second task whose standard input or standard output is one of the two ends. Week 10's fork gives the child its own fd table, copied by `dup_fd`, with each live descriptor holding another reference on the same `struct file`. Exec keeps that table, except indexes marked close-on-exec. The program named by `exec` therefore inherits whatever the child left at descriptors 0, 1, and 2.

`dup2(pipefd, 1)` makes descriptor 1 refer to the same open file as `pipefd`, then the child closes the original pipe indexes so that it does not keep an extra reference. The parent's job is the matching close of the end it will not use. Those closes are the lifetime protocol. They are not optional cleanup.

```
parent                          child A (producer)          child B (consumer)
pipe() -> r, w
fork A, fork B
close(r)                        dup2(w, STDOUT_FILENO)      dup2(r, STDIN_FILENO)
close(w) after                  close(r); close(w)          close(r); close(w)
  the last byte                 exec producer               exec consumer
waitpid both
```

After `exec`, the producer's `write(1, ...)` is `pipe_write` into a pipe page, and the consumer's `read(0, ...)` is the other end of the same `pipe_inode_info`. The programs name the ring only as the integers 0 and 1.

If the parent forgets `close(w)`, `writers` never drops. The consumer sleeps in `read` after the producer has exited, because the parent still holds the write file. A child that `dup2`s the read end onto stdin and leaves the write end open has made itself a writer, so its own later read will not see end-of-file. Section 6 is that rule in `pipe_release`.

5. Three bytes, from the user buffer to the other user buffer

Take a blocking pipe, one writer, one reader, and a three-byte write of `'a'`, `'b'`, `'c'`. `pipe_write` locks the pipe. `readers` is 1, so it does not take the `SIGPIPE` path. The ring is empty, so the merge into a previous buffer is skipped. The ring is not full. The function allocates a page if `tmp_page` is empty, advances `head` under `rd_wait.lock`, and installs a `pipe_buffer` whose `ops` are `anon_pipe_buf_ops` and whose flags are `PIPE_BUF_FLAG_CAN_MERGE`. `copy_page_from_iter` copies the three bytes at offset 0. `buf->len` becomes 3. The iterator is empty, so the loop breaks, the lock drops, and because the pipe was empty the function wakes `rd_wait` with `EPOLLIN | EPOLLRDNORM`.

That wake makes a reader blocked on `rd_wait` runnable. Week 7 is the register save and restore under that sentence. The pipe mutex is not held across the sleep.

`pipe_read` then sees `head != tail`. It confirms the buffer, and `copy_page_to_iter` copies `min(buf->len, requested)` bytes into the reader's user buffer. A reader that asked for 1 byte leaves `len == 2` and `offset == 1` and does not advance `tail`. A reader that asked for 8 bytes copies 3, sees `len == 0`, releases the buffer, and increments `tail`. The return value is the number of bytes copied, 1 or 3, not the number the writer passed and not a record header.

Both copies are kernel copies through a pipe page. The user's `write` never stores into the consumer's address space. Day 6 is what changes when a load names the page through the consumer's own page tables.

6. End-of-file is "no writer remains"

An empty pipe and a closed pipe look the same if you only test `head == tail`. The reader needs a second fact: will any future `write` still be able to append? `pipe_readable` (lines 220–227) answers the wait. It loads `head`, `tail`, and `writers` with `READ_ONCE`, because it runs without the mutex, and it returns true when the ring is non-empty or when `writers` is zero. The locked path in `pipe_read` makes the decision that becomes the syscall result:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/pipe.c
 * lines: 322–348
 * function: pipe_read
 */
            if (!buf->len) {
                pipe_buf_release(pipe, buf);
                spin_lock_irq(&pipe->rd_wait.lock);
#ifdef CONFIG_WATCH_QUEUE
                if (buf->flags & PIPE_BUF_FLAG_LOSS)
                    pipe->note_loss = true;
#endif
                tail++;
                pipe->tail = tail;
                spin_unlock_irq(&pipe->rd_wait.lock);
            }
            total_len -= chars;
            if (!total_len)
                break;  /* common path: read succeeded */
            if (!pipe_empty(head, tail))    /* More to do? */
                continue;
        }

        if (!pipe->writers)
            break;
        if (ret)
            break;
        if (filp->f_flags & O_NONBLOCK) {
            ret = -EAGAIN;
            break;
        }
        __pipe_unlock(pipe);
```

The sleep itself is the next call, still inside `pipe_read`, after the comment that explains why an interrupt here can return `-ERESTARTSYS` directly: the lock is already dropped and no byte was copied.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/pipe.c
 * lines: 367–378
 * function: pipe_read
 */
        if (unlikely(was_full))
            wake_up_interruptible_sync_poll(&pipe->wr_wait, EPOLLOUT | EPOLLWRNORM);
        kill_fasync(&pipe->fasync_writers, SIGIO, POLL_OUT);

        /*
         * But because we didn't read anything, at this point we can
         * just return directly with -ERESTARTSYS if we're interrupted,
         * since we've done any required wakeups and there's no need
         * to mark anything accessed. And we've dropped the lock.
         */
        if (wait_event_interruptible_exclusive(pipe->rd_wait, pipe_readable(pipe)) < 0)
            return -ERESTARTSYS;
```

If the ring is empty and `writers` is already 0, the function breaks with `ret` still 0. The syscall returns 0. That 0 is end-of-file. If the ring is empty, a writer still exists, and the file is blocking, the reader drops the mutex and sleeps on `rd_wait` until `pipe_readable` becomes true. If the

file is `O_NONBLOCK`, it returns `-EAGAIN` instead of sleeping. `-EAGAIN` means "empty, and a writer still exists." `0` means "empty, and no writer remains." A program that treats both as "try again" will spin after the writer has gone. A program that treats both as end-of-file will stop while the writer is merely slow.

`pipe_release` is what drives `writers` to 0. It runs when the write end's `struct file` loses its last reference.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/pipe.c
 * lines: 711–729
 * function: pipe_release
 */
static int
pipe_release(struct inode *inode, struct file *file)
{
    struct pipe_inode_info *pipe = file->private_data;

    __pipe_lock(pipe);
    if (file->f_mode & FMODE_READ)
        pipe->readers--;
    if (file->f_mode & FMODE_WRITE)
        pipe->writers--;

    /* Was that the last reader or writer, but not the other side? */
    if (!pipe->readers != !pipe->writers) {
        wake_up_interruptible_all(&pipe->rd_wait);
        wake_up_interruptible_all(&pipe->wr_wait);
        kill_fasync(&pipe->fasync_readers, SIGIO, POLL_IN);
        kill_fasync(&pipe->fasync_writers, SIGIO, POLL_OUT);
    }
    __pipe_unlock(pipe);
```

The condition `!pipe->readers != !pipe->writers` is true when one side has hit zero and the other has not. Closing the last write reference wakes every reader. A reader blocked in the wait above sees `pipe_readable` true because `writers` is now 0, reacquires the mutex, finds the ring empty, and returns 0. Closing the last read reference wakes every writer, which is the setup for the next section.

Data already in the ring is still readable after the writers have gone. `pipe_read` copies those bytes first. Only when the ring is empty does the `!pipe->writers` test return 0. End-of-file is ordered after the last byte, not instead of it.

7. No readers: the signal and the error are both generated

The matching failure on the write side is a pipe that will never be read again. `pipe_write` checks before copying anything:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/pipe.c
 * lines: 414–436
 * function: pipe_write
 */
static ssize_t
pipe_write(struct kiocb *iocb, struct iov_iter *from)
{
    struct file *filp = iocb->ki_filp;
    struct pipe_inode_info *pipe = filp->private_data;
    unsigned int head;
    ssize_t ret = 0;
    size_t total_len = iov_iter_count(from);
    ssize_t chars;
    bool was_empty = false;
    bool wake_next_writer = false;

    /* Null write succeeds. */
    if (unlikely(total_len == 0))
        return 0;

    __pipe_lock(pipe);

    if (!pipe->readers) {
        send_sig(SIGPIPE, current, 0);
        ret = -EPIPE;
        goto out;
    }
```

`send_sig(SIGPIPE, current, 0)` posts the signal, and the function sets `-EPIPE`. Both happen. Week 10's delivery decides what the task sees. The default disposition terminates the process, so the C statement after `write` never runs. `SIG_IGN` or a blocked signal lets `write` return `-1` with `errno == EPIPE`.

The same check repeats inside the copy loop (lines 477–482), because the writer drops the mutex while it waits for space. `send_sig` still runs. If some bytes are already copied, `ret` stays the positive short count and is not replaced with `-EPIPE`. A writer that ignored `SIGPIPE` must honor that count and expect the next `write` to return `EPIPE`. A writer that left the default disposition can die on this path after a partial copy, because the signal is posted either way. `EPIPE` is the syscall errno. `SIGPIPE` is the signal beside it. Day 7 puts both in one report.

8. Backpressure is a sleep on `wr_wait`

The bound is enforced in the same loop. When `pipe_full` is true and the file is blocking, the writer wakes any reader that might be waiting on an empty pipe, then sleeps:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/pipe.c
 * lines: 546–571
 * function: pipe_write
 */
        /* Wait for buffer space to become available. */
        if (filp->f_flags & O_NONBLOCK) {
            if (!ret)
                ret = -EAGAIN;
            break;
        }
        if (signal_pending(current)) {
            if (!ret)
                ret = -ERESTARTSYS;
            break;
        }

        /*
         * We're going to release the pipe lock and wait for more
         * space. We wake up any readers if necessary, and then
         * after waiting we need to re-check whether the pipe
         * become empty while we dropped the lock.
         */
        __pipe_unlock(pipe);
        if (was_empty)
            wake_up_interruptible_sync_poll(&pipe->rd_wait, EPOLLIN | EPOLLRDNORM);
        kill_fasync(&pipe->fasync_readers, SIGIO, POLL_IN);
        wait_event_interruptible_exclusive(pipe->wr_wait, pipe_writable(pipe));
        __pipe_lock(pipe);
        was_empty = pipe_empty(pipe->head, pipe->tail);
        wake_next_writer = true;
```

`pipe_writable` is the mirror of `pipe_readable`: the ring is not full, or `readers` is zero. The exclusive wait wakes one sleeper to take the space a reader just freed. `pipe_read` remembers `was_full` and, if it frees a slot, wakes `wr_wait` with `EPOLLOUT | EPOLLWRNORM`.

The waiting task leaves the runqueue the same way a futex waiter does in Week 11. Bytes already copied stay in the ring. A nonblocking write that has copied nothing returns `-EAGAIN`; one that has copied some pages returns that short count.
`wait_event_interruptible_exclusive` rechecks the predicate after publishing the waiter, which is the lost-wakeup close from the condition-variable lesson. A full pipe is a consumer that is behind. The `PIPE_MIN_DEF_BUFFERS` comment at the top of `fs/pipe.c` names the other failure: a one-slot pipe blocks a writer that still owes the pipe a token, which is how GNU make's jobserver uses one.

9. `PIPE_BUF` is an atomicity bound

The uapi header names the constant applications are told to rely on.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: include/uapi/linux/limits.h
 * lines: 5–17
 * function: (macros)
 */
#define NR_OPEN         1024

#define NGROUPS_MAX    65536    /* supplemental group IDs are available */
#define ARG_MAX       131072    /* # bytes of args + environ for exec() */
#define LINK_MAX         127    /* # links a file may have */
#define MAX_CANON        255    /* size of the canonical input queue */
#define MAX_INPUT        255    /* size of the type-ahead buffer */
#define NAME_MAX         255    /* # chars in a file name */
#define PATH_MAX        4096    /* # chars in a path name including nul */
#define PIPE_BUF        4096    /* # bytes in atomic write to a pipe */
#define XATTR_NAME_MAX   255    /* # chars in an extended attribute name */
#define XATTR_SIZE_MAX 65536    /* size of an extended attribute value (64k) */
#define XATTR_LIST_MAX 65536    /* size of extended attribute namelist (64k) */
```

The comment says atomic write. It does not say message. `fs/pipe.c` never compares a transfer against the identifier `PIPE_BUF`. The write path holds `__pipe_lock` across the copy and drops it only to sleep in section 8. Two writers cannot interleave while one holds the mutex. A write that does not fit publishes what it copied, releases the mutex, and continues after the wake, so the other writer can append between the two halves.

Readers see bytes in order, with no marker where one `write` ended. `AAAA` from one writer and `BBBB` from another can return as one `read` of eight or as a `read` of three plus a `read` of five. A length prefix, a newline, or some other frame has to be in the bytes if the reader needs records.

`O_DIRECT` on the write end is a different mechanism. `is_packetized` tests that flag, and a new buffer then gets `PIPE_BUF_FLAG_PACKET` instead of `PIPE_BUF_FLAG_CAN_MERGE`. `pipe_read` returns a packet buffer as one result and clears `buf->len`. Ordinary pipes merge small writes onto the tail of the previous page. Merging is the opposite of framing. Use `PIPE_BUF` when two racing writers must not cut a record in half, keep each record at most that size, and still find the record boundary in the bytes. Do not read the constant as "every `write` becomes one `read`."

10. ToyOS has the queue and not the protocol

Week 8 implements `pipe_data[64]`. A write succeeds only when the whole request fits. An empty read returns zero immediately. There is one global pipe, fixed descriptors 3 and 4, no `fork`, no `dup2`, no wait queue, no `EPIPE`, and no `SIGPIPE`. This lesson splits that zero into `-EAGAIN` or a sleep while a writer exists, and `0` only after the last writer releases the file. Week 9's VFS is why the queue hangs on an inode. `pipefs_dname` prints `pipe:[inode]`, which is what `/proc/PID/fd/N` shows for either end.

The comparison worth keeping:

```
ToyOS pipe                          this tree's pipe
64-byte linear buffer               ring of pages, default 16 slots
empty read returns 0 at once        empty read sleeps, or -EAGAIN, or 0 at EOF
no writer-gone state                writers hits 0 in pipe_release
no broken-pipe signal               send_sig(SIGPIPE) and -EPIPE
one global pipe, fds 3 and 4        two struct file objects, any free indexes
no second process                   fork copies the files, dup2 aims them
```

11. A pipeline small enough to trace by hand

The following is the protocol from sections 4, 6, and 7 in one process tree. It is ordinary POSIX C. It does not open the kernel objects except through the syscalls those sections implement.

```
int r, w, fds[2], status;
char buf[8];
ssize_t n;
pid_t child;

if (pipe(fds) != 0)
    return 1;
r = fds[0];
w = fds[1];
child = fork();
if (child == 0) {
    close(w);
    if (dup2(r, STDIN_FILENO) < 0)
        _exit(127);
    close(r);
    n = read(STDIN_FILENO, buf, sizeof buf);
    /* n == 3 for the write below; a later read returns 0 */
    _exit(n < 0 ? 127 : 0);
}
close(r);
if (write(w, "abc", 3) != 3)
    return 1;
close(w);                 /* last write reference: child observes EOF */
if (waitpid(child, &status, 0) < 0)
    return 1;
```

`pipe` enters `do_pipe2`. `fork` duplicates the files. The child points descriptor 0 at the read end and drops the write end. The parent drops the read end, writes `"abc"`, and closes the write end. The child's `read` returns 3, and a second `read` returns 0 because `pipe_release` drove `writers` to 0. `waitpid` collects the child.

Leave the write end open in the child and also skip the parent's `close(r)`, and `writers` stays nonzero after the parent closes `w`. The child's second `read` sleeps. `strace` shows `read` not returning, and `/proc/PID/fd` still lists the write-end `pipe:[inode]`. Close the read end in the child before the parent writes, and the parent dies on the default `SIGPIPE`. Ignore the signal and the same `write` returns `-1` with `errno == EPIPE`.

12. Mastery: Explain · Draw · Observe · Build

The exercise is ../challenges/day-004-pipes.sh. Run it from the week directory after you can predict, for a blocking pipe, what a `read` returns on empty-with-writer, empty-with-no-writer, and a short user buffer.

Explain

- Why the bytes of a pipe live in kernel pages rather than in either task's address space, and which two syscalls move them across the boundary.
- What `create_pipe_files` puts in `res[0]` and `res[1]`, and why `readers` and `writers` start at 1 even though `files` starts at 2.
- Why a parent that forks a consumer and forgets to close its own copy of the write end prevents the child from ever seeing end-of-file.
- What `pipe_write` does when `readers` is 0, and which disposition of `SIGPIPE` lets the caller observe `EPIPE`.
- Why two successful four-byte writes can come back as one eight-byte read, and what `PIPE_BUF` guarantees instead of a record boundary.

Draw

Draw one `pipe_inode_info` with a ring of four slots, `head`, `tail`, and the page each occupied slot points at. Mark `rd_wait` and `wr_wait`. Under it, draw two tasks after `fork` and `dup2`: which descriptor indexes share which `struct file`, and which `close` calls drop the last reference. Then draw the full-pipe case: writer off the runqueue, reader's `pipe_read` advancing `tail`, wakeup on `wr_wait`.

Observe

Run the wrapper. Record `uname -r`, the compiler, and the tree id if you opened the sources. In the output, point at the read that returns 0 and write down which close made `writers` hit 0. If you trace a blocking writer against a slow reader, name the syscall that does not return until the reader has consumed a slot.

Build

Write a one-page account of a single byte in a two-process pipeline: the `write` buffer, the pipe page and slot index, the `read` buffer, the `close` that produces end-of-file, and the `waitpid` that reaps the child. Include the sentence you will stand on in a report: `PIPE_BUF` bounds interleaving of concurrent writers, and the byte stream still has no frames unless the bytes themselves contain them.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Pipes

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 12-linux-io-and-readiness/challenges/day-004-pipes.sh
```

- View lab source on GitHub

Day 5 — `select`, `poll`, `epoll`, and Readiness

Target time: approximately 3 hours

> Readiness is a promise that the next I/O call on that descriptor can make progress now. It is not a count of bytes, and it is not a statement that the operation has finished.

Day 4's pipe blocks inside one `read` or one `write`. That fits a task with one source. It does not fit a task that must take the next byte from whichever of several files becomes readable first. A thread blocked in `pipe_read` on source A cannot observe source B. One thread per source spends a stack, a `task_struct`, and a wakeup on each idle source, the context-switch cost from Week 7 multiplied by sources that had nothing to say.

Readiness inverts the wait. The task sleeps once, on "which of these files would not block," and the kernel wakes it when any changes. The excerpts are Linux 5.10.134 at `c9b2385d6793`. `select`, `poll`, and `epoll` differ in where the interest set lives between calls, and in what the task does after it wakes.

1. Why "readable" cannot mean "done"

A blocking `read` of a pipe has three honest results, from Day 4. A positive count means that many bytes were copied. `0` means no writer remains. `-EAGAIN`, on a nonblocking file, means the ring is empty and a writer still exists. None of those results is known at the moment the task decides to wait. The kernel can know a weaker fact: a call made right now would not go to sleep.

`pipe_poll` computes that fact from the same counters `pipe_read` uses. A read end contributes `EPOLLIN | EPOLLRDNORM` when the ring is not empty, and `EPOLLHUP` when `writers` is zero and the writer's counter has moved. A write end contributes `EPOLLOUT | EPOLLWRNORM` when the ring is not full, and `EPOLLERR` when `readers` is zero. Those bits mean "the corresponding syscall has a chance to return without waiting." They do not mean "N bytes are buffered," and they do not mean "the peer has finished."

A pipe with one byte is readable, and a `read` of 4096 bytes returns 1. A pipe whose last writer has closed is also readable, and `read` returns 0. A socket that just became writable can still return a short count or `EAGAIN` if the window closes between the poll and the write. The wakeup is a hint. The consuming `read` or `write` rechecks the predicate, the same shape as the condition-variable loop in Week 11.

```
readiness:  "a read/write/accept issued now can progress"
the call:   copies bytes, or returns 0, or returns EAGAIN, or errors
neither:    "the transfer of the size you hoped for has completed"
```

`epoll_wait` returning 1 means one ready-list entry was copied out. The byte count is what the following `read` returns. Treating the wait count as a byte count, or stopping after one `read`, is a different protocol from the one this kernel implements.

2. Sleeping inside one file does not scale to a set

The syscall path is still the one from Week 8. User registers enter the kernel, the kernel may set the task to `TASK_INTERRUPTIBLE` and call the scheduler, and a later wakeup restores those registers so the syscall can return. Week 11's threads do not change the shape. A second thread blocked in `read` is a second task on a wait queue. That is a fine design when the threads have other work and the blocking call is the synchronization. It is a poor design when the threads exist only to wait, because each one pays a stack and a wakeup for a source that is usually idle.

The three multiplexers below share one structure, visible in `do_select` and `do_poll`:

1. Build a kernel-side description of the caller's interest.
2. Ask each file's `poll` method, which both reports the current bits and, the first time through, registers a wait-queue entry.
3. If any file is already ready, copy the result out and return.
4. Otherwise sleep until a wakeup or a timeout, then scan again.

`epoll` keeps the interest set and the wait-queue entries across calls, so step 1 is an `epoll_ctl` done once, and the sleep in `epoll_wait` waits on a single ready list instead of scanning every file.

3. `select` copies bitmaps, and the bitmap has a historical width

The uapi header in this tree fixes the width of the kernel's `fd_set`-shaped object:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: include/uapi/linux/posix_types.h
 * lines: 18–30
 * function: (macros and __kernel_fd_set)
 */
/*
 * This macro may have been defined in <gnu/types.h>. But we always
 * use the one here.
 */
#undef __FD_SETSIZE
#define __FD_SETSIZE    1024

typedef struct {
    unsigned long fds_bits[__FD_SETSIZE / (8 * sizeof(long))];
} __kernel_fd_set;

/* Type of a signal handler.  */
typedef void (*__kernel_sighandler_t)(int);
```

`__FD_SETSIZE` is 1024. That is the historical userspace `fd_set` this header is willing to name. `core_sys_select` does not allocate "a 1024-bit set" blindly. It sizes six bitmaps from the caller's `n`: one incoming and one outgoing set for read, write, and exception. It copies the incoming sets from user memory with `get_fd_set`, zeroes the result sets, and only then calls `do_select`. If `n` is larger than the process's `max_fds`, `n` is clamped to `max_fds` before that copy. The kernel can therefore scan more than 1024 descriptors when the caller passes a larger `n` and a large enough buffer. The 1024-wide type is still what a program using `fd_set`

and the `FD_SET` family can name without writing past the object. Those are two different limits, and a report has to say which one it means. Lines 647–680 of `core_sys_select` are the copy itself: six regions sized by `FDS_BYTES(n)`, a stack buffer when they fit and `kvmalloc` when they do not, `get_fd_set` from the user pointers, then `do_select`.

Every `select` pays that copy in and, after `do_select` returns, a copy of the result bits out. The interest set is not stored in the kernel between calls. The caller rebuilds it, the kernel recopies it, and the kernel walks it.

`do_select` walks the bits. A set bit becomes `fdget` plus `vfs_poll`. The first time a file reports a requested condition, the result counter increments and `wait->_qproc` is cleared so later files are not asked to register a waiter that this call will throw away immediately.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/select.c
 * lines: 526–555
 * function: do_select
 */
            for (j = 0; j < BITS_PER_LONG; ++j, ++i, bit <<= 1) {
                struct fd f;
                if (i >= n)
                    break;
                if (!(bit & all_bits))
                    continue;
                mask = EPOLLNVAL;
                f = fdget(i);
                if (f.file) {
                    wait_key_set(wait, in, out, bit,
                                 busy_flag);
                    mask = vfs_poll(f.file, wait);

                    fdput(f);
                }
                if ((mask & POLLIN_SET) && (in & bit)) {
                    res_in |= bit;
                    retval++;
                    wait->_qproc = NULL;
                }
                if ((mask & POLLOUT_SET) && (out & bit)) {
                    res_out |= bit;
                    retval++;
                    wait->_qproc = NULL;
                }
                if ((mask & POLLEX_SET) && (ex & bit)) {
                    res_ex |= bit;
                    retval++;
                    wait->_qproc = NULL;
                }
```

A closed or never-opened index hits `mask = EPOLLNVAL` because `fdget` finds no file. That bit does not become a readable result. If nothing is ready, no signal is pending, and the timeout has not fired, `do_select` sleeps in `poll_schedule_timeout` with `TASK_INTERRUPTIBLE` (lines 606–608). The wait-queue entries were installed by the files' `poll` methods on the first pass, through `poll_wait`, before any of them reported ready. On wakeup the loop scans every bit again. The return value of `select` is the number of bits set in the result sets, not the number of bytes behind them. A return of 2 means two conditions, on one descriptor or on two. The

caller still has to `FD_ISSET` and then `read` or `write`. Every call pays the copy and the scan, including the calls that find nothing ready, and the bitmap has no place to hang a pointer except a side array indexed by the same integer.

4. `poll` walks a list the caller already built

`poll` takes an array of `struct pollfd`: an fd, an events mask, and a slot for revents. There is no `fd_set` and no `__FD_SETSIZE` in this path. `do_sys_poll` rejects `nfds` above `RLIMIT_NOFILE`, copies the array into a `poll_list` that starts on the stack and continues onto kmalloc'd pages, then calls `do_poll`.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/select.c
 * lines: 907–934
 * function: do_poll
 */
       for (walk = list; walk != NULL; walk = walk->next) {
           struct pollfd * pfd, * pfd_end;

           pfd = walk->entries;
           pfd_end = pfd + walk->len;
           for (; pfd != pfd_end; pfd++) {
               /*
                * Fish for events. If we found one, record it
                * and kill poll_table->_qproc, so we don't
                * needlessly register any other waiters after
                * this. They'll get immediately deregistered
                * when we break out and return.
                */
               if (do_pollfd(pfd, pt, &can_busy_loop,
                             busy_flag)) {
                   count++;
                   pt->_qproc = NULL;
                   /* found something, stop busy polling */
                   busy_flag = 0;
                   can_busy_loop = false;
               }
           }
       }
       /*
        * All waiters have already been registered, so don't provide
        * a poll_table->_qproc to them on the next loop iteration.
        */
       pt->_qproc = NULL;
```

`do_pollfd` does `fdget`, calls `vfs_poll`, masks the result with the requested events plus `EPOLLERR | EPOLLHUP`, and stores `revents`. A negative fd is skipped and contributes an empty `revents`, which is how a caller punches a hole in the array without compacting it. The sleep at the bottom of `do_poll` is `poll_schedule_timeout` with `TASK_INTERRUPTIBLE`, the same wait `do_select` uses. After the call, `do_sys_poll` writes each `revents` back to the user array.

The list is rebuilt on every `poll`, and the kernel asks every file until one is ready and `_qproc` is cleared. What `poll` buys is a denser interest set: sparse descriptors are `pollfd`s with their own masks, not a bitmap with a historical width. The scan is still linear in the number of files.

5. `epoll` stores the interest set and wakes from a ready list

`epoll_ctl` is the call that edits the interest set. `do_epoll_ctl` rejects a bad epoll descriptor or a bad target with `-EBADF`, and a file whose operations cannot poll with `-EPERM`. The operation switch is the whole interface:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/eventpoll.c
 * lines: 2264–2288
 * function: do_epoll_ctl
 */
    error = -EINVAL;
    switch (op) {
    case EPOLL_CTL_ADD:
        if (!epi) {
            epds->events |= EPOLLERR | EPOLLHUP;
            error = ep_insert(ep, epds, tf.file, fd, full_check);
        } else
            error = -EEXIST;
        break;
    case EPOLL_CTL_DEL:
        if (epi)
            error = ep_remove(ep, epi);
        else
            error = -ENOENT;
        break;
    case EPOLL_CTL_MOD:
        if (epi) {
            if (!(epi->event.events & EPOLLEXCLUSIVE)) {
                epds->events |= EPOLLERR | EPOLLHUP;
                error = ep_modify(ep, epi, epds);
            }
        } else
            error = -ENOENT;
        break;
    }
```

`ADD` of a descriptor already in the tree returns `-EEXIST`. `DEL` or `MOD` of one that is absent returns `-ENOENT`. `ADD` forces `EPOLLERR | EPOLLHUP` into the mask, so the caller cannot suppress hangup and error by forgetting them. The lookup key is the pair `(struct file *, fd)`, compared by `ep_cmp_ffd`. Two descriptor numbers that refer to different files are different items. The same file under two descriptor numbers is also two items, because the fd is part of the key.

`ep_insert` allocates an `epitem`, hangs it on the file's `f_ep_links`, and inserts it into the epoll instance's red-black tree. It then polls once, which is what installs the callback:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/eventpoll.c
 * lines: 1548–1579
 * function: ep_insert
 */
    /* Initialize the poll table using the queue callback */
    epq.epi = epi;
    init_poll_funcptr(&epq.pt, ep_ptable_queue_proc);

    /*
     * Attach the item to the poll hooks and get current event bits.
     * We can safely use the file* here because its usage count has
     * been increased by the caller of this function. Note that after
     * this operation completes, the poll callback can start hitting
     * the new item.
     */
    revents = ep_item_poll(epi, &epq.pt, 1);

    /*
     * We have to check if something went wrong during the poll wait queue
     * install process. Namely an allocation for a wait queue failed due
     * high memory pressure.
     */
    error = -ENOMEM;
    if (epi->nwait < 0)
        goto error_unregister;

    /* We have to drop the new item inside our item list to keep track of it */
    write_lock_irq(&ep->lock);

    /* record NAPI ID of new item if present */
    ep_set_busy_poll_napi_id(epi);

    /* If the file is already "ready" we drop it inside the ready list */
    if (revents && !ep_is_linked(epi)) {
        list_add_tail(&epi->rdllink, &ep->rdllist);
        ep_pm_stay_awake(epi);
```

`ep_item_poll` calls `vfs_poll`. For a pipe that is `pipe_poll`. `poll_wait` calls the poll table's `_qproc` when one is set. The qproc from `ep_insert` is `ep_ptable_queue_proc`, which points an `eppoll_entry` at `ep_poll_callback` and adds it to the file's wait queue. A pipe read end passes `&pipe->rd_wait`, the queue Day 4's `pipe_write` wakes. If the file is already ready, the item is linked onto `rdllist` and any sleeper on `ep->wq` is woken. The red-black tree is the interest set. `rdllist` is what a callback has marked and `epoll_wait` has not finished with.

`do_epoll_wait` checks the caller's buffer, rejects a descriptor that is not an epoll file, and then sleeps inside `ep_poll`:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/eventpoll.c
 * lines: 2336–2364
 * function: do_epoll_wait
 */
    /* The maximum number of event must be greater than zero */
    if (maxevents <= 0 || maxevents > EP_MAX_EVENTS)
        return -EINVAL;

    /* Verify that the area passed by the user is writeable */
    if (!access_ok(events, maxevents * sizeof(struct epoll_event)))
        return -EFAULT;

    /* Get the "struct file *" for the eventpoll file */
    f = fdget(epfd);
    if (!f.file)
        return -EBADF;

    /*
     * We have to check that the file structure underneath the fd
     * the user passed to us _is_ an eventpoll file.
     */
    error = -EINVAL;
    if (!is_file_epoll(f.file))
        goto error_fput;

    /*
     * At this point it is safe to assume that the "private_data" contains
     * our own data structure.
     */
    ep = f.file->private_data;

    /* Time to fish for events ... */
    error = ep_poll(ep, events, maxevents, timeout);
```

`ep_poll` sets `TASK_INTERRUPTIBLE`, rechecks `ep_events_available` under `ep->lock`, and only then calls `__add_wait_queue_exclusive` on `ep->wq` (lines 1921–1935). That is the lost-wakeup close: recheck under the waker's lock before sleeping. A signal already pending returns `-EINTR`. The callback runs in the waker's context and must not sleep, which is why the comment at the top of `fs/eventpoll.c` makes `ep->lock` a rwlock. The usual path links the item if it is not already linked, then wakes `ep->wq`:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/eventpoll.c
 * lines: 1249–1280
 * function: ep_poll_callback
 */
   if (READ_ONCE(ep->ovflist) != EP_UNACTIVE_PTR) {
       if (chain_epi_lockless(epi))
           ep_pm_stay_awake_rcu(epi);
   } else if (!ep_is_linked(epi)) {
       /* In the usual case, add event to ready list. */
       if (list_add_tail_lockless(&epi->rdllink, &ep->rdllist))
           ep_pm_stay_awake_rcu(epi);
   }

   /*
    * Wake up ( if active ) both the eventpoll wait list and the ->poll()
    * wait list.
    */
   if (waitqueue_active(&ep->wq)) {
       if ((epi->event.events & EPOLLEXCLUSIVE) &&
                   !(pollflags & POLLFREE)) {
           switch (pollflags & EPOLLINOUT_BITS) {
           case EPOLLIN:
               if (epi->event.events & EPOLLIN)
                   ewake = 1;
               break;
           case EPOLLOUT:
               if (epi->event.events & EPOLLOUT)
                   ewake = 1;
               break;
           case 0:
               ewake = 1;
               break;
           }
       }
       wake_up(&ep->wq);
   }
```

If `epoll_wait` is copying events out, `ovflist` is the overflow chain. Events that arrive in that window are queued there and reattached, so a callback during `copy_to_user` does not lose the edge.

```
interest (RB tree of epitem)          ready (rdllist)
  built by epoll_ctl ADD                built by ep_poll_callback
  survives across epoll_wait            consumed by ep_send_events
  one node per (file, fd)               a node is linked only when ready
```

6. Trace: pipe write, wakeup, `epoll_wait`, read

One byte is written to a pipe whose read end is already in an epoll set with `EPOLLIN`. Registration already happened. `epoll_ctl(ADD)` reached `ep_insert`, which polled the pipe. At lines 652–664 `pipe_poll` stores `poll_usage` and calls `poll_wait` on `rd_wait` before it reads `head` and `tail`, so a wake between the test and the sleep still finds a waiter. The waiter, if the ring was empty, is `TASK_INTERRUPTIBLE` on `ep->wq`, not on the pipe. Its registers sit at the syscall boundary from Week 8. The core can run the writer.

The writer enters `pipe_write`, copies the byte into a pipe page, and wakes the read-side queue. The wake is unconditional once `poll_usage` is set, even if the pipe was not empty:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/pipe.c
 * lines: 578–591
 * function: pipe_write
 */
    /*
     * If we do do a wakeup event, we do a 'sync' wakeup, because we
     * want the reader to start processing things asap, rather than
     * leave the data pending.
     *
     * This is particularly important for small writes, because of
     * how (for example) the GNU make jobserver uses small writes to
     * wake up pending jobs
     *
     * Epoll nonsensically wants a wakeup whether the pipe
     * was already empty or not.
     */
    if (was_empty || pipe->poll_usage)
        wake_up_interruptible_sync_poll(&pipe->rd_wait, EPOLLIN | EPOLLRDNORM);
```

`wake_up_interruptible_sync_poll` walks `rd_wait`. The entry installed for epoll has function `ep_poll_callback`. The callback links the `epitem` onto `rdllist` and calls `wake_up(&ep->wq)`. That second wake is the one that makes the `epoll_wait` task runnable. The scheduler restores its registers. `ep_poll` reaches `ep_send_events`, which polls the pipe once more and copies an `epoll_event` to the user buffer. The syscall returns 1.

Only now does the user task call `read` on the pipe. That is `pipe_read`. It copies however many bytes are in the ring, up to the buffer it was given, and returns that count. If the writer has closed and the ring is empty, it returns 0. If another task drained the byte between the callback and the `read`, a nonblocking read returns `-EAGAIN`. The ready event was still a true statement about the past. It was not a reservation of the byte.

```
pipe_write
  copy byte into pipe page
  wake_up rd_wait          ---> ep_poll_callback
                                  link epitem on rdllist
                                  wake_up ep->wq
                                       |
epoll_wait task                        v
  was TASK_INTERRUPTIBLE on ep->wq
  ep_send_events copies EPOLLIN to user
  syscall returns 1
user read()
  pipe_read copies the byte, returns the count
```

A trace that stops at `epoll_wait` returning 1 has not observed the byte. A trace that stops at `read` returning 1 has.

7. Level trigger, edge trigger, and the `EAGAIN` drain

After an event is copied out, `ep_send_events_proc` decides whether the item stays interesting. Lines 1750–1762 re-poll with a null qproc, so this poll observes and does not register another waiter, then `__put_user` the event. A failed `put_user` links the item back and can return `-EFAULT`. The branch after a successful copy is the level-versus-edge rule:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/eventpoll.c
 * lines: 1764–1779
 * function: ep_send_events_proc
 */
       if (epi->event.events & EPOLLONESHOT)
           epi->event.events &= EP_PRIVATE_BITS;
       else if (!(epi->event.events & EPOLLET)) {
           /*
            * If this file has been added with Level
            * Trigger mode, we need to insert back inside
            * the ready list, so that the next call to
            * epoll_wait() will check again the events
            * availability. At this point, no one can insert
            * into ep->rdllist besides us. The epoll_ctl()
            * callers are locked out by
            * ep_scan_ready_list() holding "mtx" and the
            * poll callback will queue them in ep->ovflist.
            */
           list_add_tail(&epi->rdllink, &ep->rdllist);
           ep_pm_stay_awake(epi);
```

Default mode is level trigger: `EPOLLET` is absent, so the item goes back on `rdllist`. The next `epoll_wait` reports it again until a poll shows the condition gone. A short `read` is fine. The following wait notices the leftover bytes.

`EPOLLET` does not requeue. The next report requires a new callback. `pipe_write` wakes on every write once `poll_usage` is set, so a later write is a new edge. A short `read` that leaves bytes, with no further write, produces none. `epoll_wait` sleeps and the bytes sit in the ring. The repair is to read until the file would block, which means `O_NONBLOCK` so the drain sees `-EAGAIN` instead of sleeping inside `pipe_read` while another descriptor in the set is ready. `-EAGAIN` is "drained." `0` is "last writer closed." The loop stops on `EAGAIN` and returns to `epoll_wait`. It ends the connection on `0`.

`EPOLLONESHOT` clears the interest bits, leaving `EP_PRIVATE_BITS` (`EPOLLWAKEUP`, `EPOLLONESHOT`, `EPOLLET`, `EPOLLEXCLUSIVE`). `ep_poll_callback` treats a mask with no poll bits as disabled until `EPOLL_CTL_MOD` puts `EPOLLIN` back. Forgetting the `MOD` looks like an edge-triggered stall: `epoll_wait` stays quiet, and the pipe still holds bytes.

8. Close drops the interest when the file's last reference drops

The epoll instance does not own the caller's descriptor number. As `ep_insert`'s comment says, the caller already raised the file's usage count. When that file's last reference drops, `__fput` in `fs/file_table.c` calls `eventpoll_release` before the file's own `release` (lines 267–272). `eventpoll_release` returns if `f_ep_links` is empty. Otherwise `eventpoll_release_file` walks the links and `ep_remove`s each item. A `dup` keeps the `struct file`, so the `epitem`

remains, still keyed by the fd passed to `EPOLL_CTL_ADD`. Closing the epoll file itself is `ep_eventpoll_release`, a different path that frees the interest set because the instance is going away.

`EPOLL_CTL_DEL` drops one `(file, fd)` key while the file stays open. Closing the last descriptor that refers to the file also drops the interest, because `__fput` runs. Closing a duplicate, while another descriptor still refers to the same file, does not. Adding the same key twice returns `-EEXIST`. After close-and-reuse, the new `struct file` does not match the old item: `ep_cmp_ffd` compares the file pointer first.

9. A loop that matches the kernel's rule

This is the edge-triggered pipe reader the branch in section 7 describes. Level trigger can stop after one `read`. Edge trigger cannot.

```
struct epoll_event ev, out;
int ep, n;
char buf[256];

if (fcntl(rfd, F_SETFL, O_NONBLOCK) < 0)
    return -1;
ep = epoll_create1(EPOLL_CLOEXEC);
if (ep < 0)
    return -1;
ev.events = EPOLLIN | EPOLLET;
ev.data.fd = rfd;
if (epoll_ctl(ep, EPOLL_CTL_ADD, rfd, &ev) < 0)
    return -1;

for (;;) {
    n = epoll_wait(ep, &out, 1, -1);
    if (n < 0) {
        if (errno == EINTR)
            continue;
        return -1;
    }
    /* n == 1 means "this fd can progress", not "1 byte is waiting". */
    for (;;) {
        ssize_t r = read(rfd, buf, sizeof buf);
        if (r > 0)
            continue;             /* r bytes; keep draining */
        if (r == 0)
            return 0;             /* last writer released the file */
        if (errno == EINTR)
            continue;
        if (errno == EAGAIN)
            break;                /* level is gone; wait for the next edge */
        return -1;
    }
}
```

A writer puts 300 bytes into an empty pipe and closes. `epoll_wait` returns 1. The reads return 256, then 44, then 0, and the loop ends. If the writer stays open, the third `read` returns `-1`/`EAGAIN`, the inner loop breaks, and `epoll_wait` sleeps until the next write. Forget `O_NONBLOCK`

and that third `read` blocks inside `pipe_read`, so a second pipe in the same set is ignored until it returns. `select` on the same pipe is still the copy and the scan. A positive result still allows a short `read`. Timeout zero sets `timed_out` up front and does not sleep.

10. Mastery: Explain · Draw · Observe · Build

The exercise is ../challenges/day-005-readiness.sh. Predict, before you run it, which call returns a count of ready conditions and which later call returns a count of bytes.

Explain

- Why a task blocked in `pipe_read` on one descriptor cannot observe a second descriptor, and what one shared sleep replaces.
- What a readiness bit promises about the next `read` or `write`, and which two pipe results (`0` and `-EAGAIN`) a readable pipe can still produce.
- Why every `select` copies bitmaps and scans them, and which object in this tree is 1024 descriptors wide.
- Where an epoll interest set lives between `epoll_wait` calls, and which function links an item onto `rdllist`.
- Why an `EPOLLET` reader that stops after one short `read` can sleep in `epoll_wait` with bytes still in the pipe, and what `O_NONBLOCK` plus `EAGAIN` changes about that loop.

Draw

Draw three columns for one pipe read end: `select`'s bit in a copied bitmap, `poll`'s `pollfd`, and epoll's `epitem` in the red-black tree plus a second box for `rdllist`. Then draw the section 6 trace as five arrows: `pipe_write`, `rd_wait`, `ep_poll_callback`, `ep->wq`, `read`. Mark which arrow carries a byte count and which arrows carry only a wake.

Observe

Run the wrapper. Record `uname -r` and the tree id if you opened sources. In the trace, point at `epoll_wait`'s return and at the following `read`'s return, and write both numbers down. If you have a level-triggered run and an edge-triggered run, say which one reports the pipe again after a short read with no further write.

Build

Write the drain loop from section 9 in your own names and annotate each return: ready-count, byte-count, end-of-file, and would-block. State the `EPOLLONESHOT` rule in one sentence: delivery clears the interest bits until `EPOLL_CTL_MOD` puts them back. Keep Day 4's pipe ring in the picture, so the ready event and the byte never get collapsed into one number.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Readiness

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 12-linux-io-and-readiness/challenges/day-005-readiness.sh
```

- View lab source on GitHub

Day 6 — `mmap`, Shared Mappings, and Data Movement

Target time: approximately 3 hours

> `mmap` does not copy the file. It publishes a virtual range whose page-table entries, filled in by faults, name pages in the page cache. After that, the CPU's ordinary loads and stores are the I/O.

`read` and `write` move bytes by copying. Day 4's pipe does it twice: once from the writer's buffer into a kernel page, once from that page into the reader's buffer. A file `read` does the same shape through the page cache. Week 9 drew that chain and stopped at the boundary ToyOS does not cross. Today the chain is real, and there is a second way to finish it. Instead of asking the kernel to copy a range into a buffer you allocated, you ask the kernel to add a VMA. The first load or store of a missing page faults. The fault handler finds or reads the cache page and installs a PTE. The faulting instruction is then restarted. The load that faulted is the load that observes the file byte.

Neither path is the fast one for every program. The copy path pays a memory copy and gets a simple lifetime: the buffer is anonymous memory, the call returns a count, and a short file returns a short count. The mapping path pays faults, TLB entries, and a signal if the file shrinks under you, and it gets a path where a later load does not enter the kernel at all. The excerpts are Linux 5.10.134 at tree `c9b2385d6793`.

1. Why a load should be able to name a file byte

A userspace buffer and a page-cache page are both physical memory. Week 7's page tables are what decide which virtual address names which physical page. `read` keeps those names apart. The cache page has a kernel virtual address, or is mapped temporarily, and `copy_to_user` stores into the user buffer's page. The user program never holds a virtual address for the cache page. That is easy to reason about, and it is mandatory when the consumer is not allowed to keep the page, or when the data is a pipe buffer that has no file offset.

It is a tax when the program's next act is to load those same bytes with ordinary instructions. The copy exists so the loads have something to hit. If the PTE can name the cache page directly, the copy is unnecessary for every byte the program actually touches, and the bytes it never touches are never copied. That is the reason `mmap` exists. The cost moves to the first reference, which becomes a page fault, and to every later protection or lifetime change, which becomes a TLB shootdown and possibly a signal rather than an errno.

The CPU does not know the name "page cache." It knows a load or a store, a virtual address, and a translation. Week 2 is that instruction. Week 6 is the trap that runs when the translation is missing or forbidden. The kernel's job on that trap is to make the translation true and return to the faulting instruction, or to deliver a signal if it cannot.

2. `mmap` builds a VMA and returns

On x86-64 the syscall checks the file offset before it will call the common path. A byte offset that is not a multiple of the page size is rejected. The shift that follows is what `ksys_mmap_pgoff` receives as a page offset.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: arch/x86/kernel/sys_x86_64.c
 * lines: 86–98
 * function: mmap (SYSCALL_DEFINE6)
 */
SYSCALL_DEFINE6(mmap, unsigned long, addr, unsigned long, len,
        unsigned long, prot, unsigned long, flags,
        unsigned long, fd, unsigned long, off)
{
    long error;
    error = -EINVAL;
    if (off & ~PAGE_MASK)
        goto out;

    error = ksys_mmap_pgoff(addr, len, prot, flags, fd, off >> PAGE_SHIFT);
out:
    return error;
}
```

`ksys_mmap_pgoff` turns a descriptor into a `struct file`, or refuses to. `MAP_ANONYMOUS` skips the file. Anything else takes a reference with `fget` and returns `-EBADF` when the descriptor does not name one. The same `-EBADF` is what a bad pipe descriptor produces in Day 4's table lookup. The object behind a successful file mapping is the open file Week 9 already defined, not a new kind of kernel object.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: mm/mmap.c
 * lines: 1690–1707
 * function: ksys_mmap_pgoff
 */
unsigned long ksys_mmap_pgoff(unsigned long addr, unsigned long len,
                  unsigned long prot, unsigned long flags,
                  unsigned long fd, unsigned long pgoff)
{
    struct file *file = NULL;
    unsigned long retval;

    if (!(flags & MAP_ANONYMOUS)) {
        audit_mmap_fd(fd, flags);
        file = fget(fd);
        if (!file)
            return -EBADF;
        if (is_file_hugepages(file)) {
            len = ALIGN(len, huge_page_size(hstate_file(file)));
        } else if (unlikely(flags & MAP_HUGETLB)) {
            retval = -EINVAL;
            goto out_fput;
        }
```

`do_mmap` is where the range becomes a mapping. The length is rounded up to a page. A length that rounds to zero returns `-ENOMEM`. An offset that would wrap the page-count addition returns `-EOVERFLOW`. Too many VMAs returns `-ENOMEM` as well. The function then asks `get_unmapped_area` for an address if the caller did not fix one, and it turns `prot` into `vm_flags` with `calc_vm_prot_bits`. A later store is allowed only if those flags include write permission. `access_error` in `arch/x86/mm/fault.c` returns failure for a write fault when the VMA lacks `VM_WRITE` (lines 1134–1137 of that file), and `do_user_addr_fault` does not call `handle_mm_fault` in that case.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: mm/mmap.c
 * lines: 1490–1501
 * function: do_mmap
 */
    /* Careful about overflows.. */
    len = PAGE_ALIGN(len);
    if (!len)
        return -ENOMEM;

    /* offset overflow? */
    if ((pgoff + (len >> PAGE_SHIFT)) < pgoff)
        return -EOVERFLOW;

    /* Too many mappings? */
    if (mm->map_count > sysctl_max_map_count)
        return -ENOMEM;
```

For a file, `MAP_SHARED` and `MAP_PRIVATE` share the checks that the file is readable, that a `mmap` file operation exists, and that the VMA is not allowed to grow like a stack. They differ in `VM_SHARED`.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: mm/mmap.c
 * lines: 1595–1618
 * function: do_mmap
 */
            /*
             * Make sure there are no mandatory locks on the file.
             */
            if (locks_verify_locked(file))
                return -EAGAIN;

            vm_flags |= VM_SHARED | VM_MAYSHARE;
            if (!(file->f_mode & FMODE_WRITE))
                vm_flags &= ~(VM_MAYWRITE | VM_SHARED);
            fallthrough;
        case MAP_PRIVATE:
            if (!(file->f_mode & FMODE_READ))
                return -EACCES;
            if (path_noexec(&file->f_path)) {
                if (vm_flags & VM_EXEC)
                    return -EPERM;
                vm_flags &= ~VM_MAYEXEC;
            }

            if (!file->f_op->mmap)
                return -ENODEV;
            if (vm_flags & (VM_GROWSDOWN|VM_GROWSUP))
                return -EINVAL;
            break;
```

`MAP_SHARED` sets `VM_SHARED | VM_MAYSHARE`, then clears `VM_MAYWRITE` and `VM_SHARED` again if the open file itself is not writable. A shared mapping of a read-only file descriptor cannot become a write into the page cache through this path. `MAP_PRIVATE` does not set `VM_SHARED`. That one bit is what `do_fault` later uses to choose a copy-on-write fault instead of a write into the cache page.

`do_mmap` finishes by calling `mmap_region`, which installs the VMA. Unless the caller set `MAP_POPULATE` (and not `MAP_NONBLOCK`) or `MAP_LOCKED`, the function does not fill PTEs. The syscall returns the start address. Every page in the range is still a hole as far as the TLB is concerned. The return of `mmap` is the publication of a permission and a file offset, not the arrival of the bytes.

`generic_file_mmap` is what a normal disk file's `mmap` operation installs as the fault ops. The connection from a VMA to `filemap_fault` is this table:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: mm/filemap.c
 * lines: 3175–3192
 * function: generic_file_vm_ops, generic_file_mmap
 */
const struct vm_operations_struct generic_file_vm_ops = {
    .fault      = filemap_fault,
    .map_pages  = filemap_map_pages,
    .page_mkwrite   = filemap_page_mkwrite,
};

/* This is used for a general mmap of a disk file */

int generic_file_mmap(struct file * file, struct vm_area_struct * vma)
{
    struct address_space *mapping = file->f_mapping;

    if (!mapping->a_ops->readpage)
        return -ENOEXEC;
    file_accessed(file);
    vma->vm_ops = &generic_file_vm_ops;
    return 0;
}
```

A mapping whose address space cannot `readpage` fails here with `-ENOEXEC`. Pipes are not this path. A pipe has no page-cache offset for a byte in the ring. Mapping a pipe is a different question from mapping a regular file, and this lesson's fault trace is the regular file.

3. The fault is the data movement

The instruction that touches the new range is an ordinary load or store. The TLB has no entry. The walker from Week 7 finds no present PTE, or a PTE whose permission is short. The CPU raises a page fault. The saved frame is the one Week 6 built a uniform version of. On x86 the fault address arrives beside the registers. The handler does not emulate the load. It installs a translation and returns to the same instruction pointer.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: arch/x86/mm/fault.c
 * lines: 1426–1440
 * function: do_user_addr_fault
 */
retry:
    vma = lock_mm_and_find_vma(mm, address, regs);
    if (unlikely(!vma)) {
        bad_area_nosemaphore(regs, hw_error_code, address);
        return;
    }

    /*
     * Ok, we have a good vm_area for this memory access, so
     * we can handle it..
     */
    if (unlikely(access_error(hw_error_code, vma))) {
        bad_area_access_error(regs, hw_error_code, address, vma);
        return;
    }
```

Line 1455 is the call that continues a permitted fault: `fault = handle_mm_fault(vma, address, flags, regs)`. The comment immediately above it says a `VM_FAULT_RETRY` means `mmap_lock` was dropped, so the handler must not assume the VMA pointer is still valid on that return.

No VMA means the address was never mapped, or was already removed by `munmap`. That path is `bad_area_nosemaphore`, a segmentation fault, not a bus error. A VMA that exists but does not allow the access is `access_error`, still before any page-cache work. Only a VMA that covers the address and permits the access reaches `handle_mm_fault`. Lines 5251–5262 count `PGFAULT` and mark the task running before this dispatch.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: mm/memory.c
 * lines: 5264–5285
 * function: handle_mm_fault
 */
    if (!arch_vma_access_permitted(vma, flags & FAULT_FLAG_WRITE,
                        flags & FAULT_FLAG_INSTRUCTION,
                        flags & FAULT_FLAG_REMOTE))
        return VM_FAULT_SIGSEGV;

    /*
     * Enable the memcg OOM handling for faults triggered in user
     * space.  Kernel faults are handled more gracefully.
     */
    if (flags & FAULT_FLAG_USER)
        mem_cgroup_enter_user_fault();

    lru_gen_enter_fault(vma);

    if (unlikely(is_vm_hugetlb_page(vma)))
        ret = hugetlb_fault(vma->vm_mm, vma, address, flags);
#ifdef CONFIG_PAGETABLE_SHARE
    else if  (unlikely(vma_is_pgtable_shared(vma)))
        ret = pgtable_share_fault(vma, address, flags);
#endif
    else
        ret = __handle_mm_fault(vma, address, flags);
```

`__handle_mm_fault` (lines 5072–5086) sets `vmf.address` to `address & PAGE_MASK` and `vmf.pgoff` to `linear_page_index`. That index is how many pages from the start of the VMA, which `do_mmap` tied to `pgoff`. The first and last byte of one page share one cache page. The function walks the page-table levels and returns `handle_pte_fault(&vmf)` at line 5160. A file-backed hole in the PTE reaches `do_fault`. The write bit and `VM_SHARED` pick the kind of fault:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: mm/memory.c
 * lines: 4675–4688
 * function: do_fault
 */
    } else if (!(vmf->flags & FAULT_FLAG_WRITE))
        ret = do_read_fault(vmf);
    else if (!(vma->vm_flags & VM_SHARED))
        ret = do_cow_fault(vmf);
    else
        ret = do_shared_fault(vmf);

    /* preallocated pagetable is unused: free it */
    if (vmf->prealloc_pte) {
        pte_free(vm_mm, vmf->prealloc_pte);
        vmf->prealloc_pte = NULL;
    }
    return ret;
}
```

A read fault calls `do_read_fault`, which calls `__do_fault`, which calls `vma->vm_ops->fault`. For the ops installed above, that function is `filemap_fault`. The comment above `__do_fault` names that function (lines 4012–4015). The call is line 4044:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: mm/memory.c
 * lines: 4037–4049
 * function: __do_fault
 */
    if (pmd_none(*vmf->pmd) && !vmf->prealloc_pte) {
        vmf->prealloc_pte = pte_alloc_one(vma->vm_mm);
        if (!vmf->prealloc_pte)
            return VM_FAULT_OOM;
        smp_wmb(); /* See comment in __pte_alloc() */
    }

    ret = vma->vm_ops->fault(vmf);
    if (unlikely(ret & (VM_FAULT_ERROR | VM_FAULT_NOPAGE | VM_FAULT_RETRY |
                VM_FAULT_DONE_COW)))
        return ret;

    if (unlikely(PageHWPoison(vmf->page))) {
```

`filemap_fault` turns the page index into a cache lookup. If the index is past `i_size`, there is no byte to name, and the fault becomes `VM_FAULT_SIGBUS`. If the page is already cached, it is used. If it is not, the fault is counted as major and the address space's `readpage` path is started by allocating a cache page.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: mm/filemap.c
 * lines: 2959–2988
 * function: filemap_fault
 */
    max_off = DIV_ROUND_UP(i_size_read(inode), PAGE_SIZE);
    if (unlikely(offset >= max_off))
        return VM_FAULT_SIGBUS;

    /*
     * Do we have something in the page cache already?
     */
    page = find_get_page(mapping, offset);
    if (likely(page) && !(vmf->flags & FAULT_FLAG_TRIED)) {
        /*
         * We found the page, so try async readahead before
         * waiting for the lock.
         */
        fpin = do_async_mmap_readahead(vmf, page);
    } else if (!page) {
        /* No page in the page cache at all */
        count_vm_event(PGMAJFAULT);
        count_memcg_event_mm(vmf->vma->vm_mm, PGMAJFAULT);
        ret = VM_FAULT_MAJOR;
        fpin = do_sync_mmap_readahead(vmf);
retry_find:
        page = pagecache_get_page(mapping, offset,
                      FGP_CREAT|FGP_FOR_MMAP,
                      vmf->gfp_mask);
        if (!page) {
            if (fpin)
                goto out_retry;
            return VM_FAULT_OOM;
        }
    }
```

When the locked page is up to date and still inside `i_size`, `filemap_fault` stores it in `vmf->page` and returns `VM_FAULT_LOCKED` (lines 3022–3030). The finish path installs a PTE and unlocks the page. `VM_FAULT_RETRY` sets `FAULT_FLAG_TRIED` and jumps back to `retry` (lines 1470–1473), because `mmap_lock` was dropped. `VM_FAULT_SIGBUS` becomes `SIGBUS` in `mm_fault_error`, which Day 7 keeps distinct from a syscall errno. The trap return restores the saved instruction pointer. The load or store runs again, the TLB fills from the new PTE, and the bytes move between the cache page and a register with no `copy_to_user`.

```
load/store
  TLB miss, page-table walk misses or denies
  page-fault trap, fault address, error code
do_user_addr_fault
  lock_mm_and_find_vma
  access_error?  -> SIGSEGV, no cache lookup
handle_mm_fault
  __handle_mm_fault: address & PAGE_MASK, linear_page_index
  do_fault -> vm_ops->fault == filemap_fault
filemap_fault
  offset >= i_size -> VM_FAULT_SIGBUS
  else find or create the page-cache page
finish: PTE points at that page
iret to the same instruction
  TLB fill, the load or store completes
```

A `read` of the same file does not take this trap. It enters on the syscall instruction, finds the same cache page, and copies into the user buffer. A `read` that filled the cache, then a fault on that offset, takes `find_get_page` and does not start I/O again. A fault that filled the cache, then a `read`, copies out of the page the fault brought in. The page cache is the meeting point.

4. Shared writes the cache page; private writes a copy

A write fault on a `MAP_SHARED` VMA takes `do_shared_fault`. The page `filemap_fault` returns is the cache page. `filemap_page_mkwrite`, the third op in `generic_file_vm_ops`, marks that page dirty so writeback can see it. Another process that has `MAP_SHARED` the same offset faults in the same cache page. A store through either mapping is a store to that page. A later `read` of the file copies those bytes out. This is the sense in which the mapping is shared: one physical page, several PTEs, the file's cache as the backing.

A write fault on a `MAP_PRIVATE` VMA takes `do_cow_fault`. Lines 4565–4584 allocate `cow_page` with `alloc_page_vma`, charge it, call `__do_fault` for the cache page, then `copy_user_highpage` onto `cow_page` and `finish_fault`. `copy_user_highpage` is the moment the private mapping stops being the file. The PTE names `cow_page`. The cache page is unlocked and dropped. A store through the private mapping does not dirty the file, and a later `read` of the file does not see it. Another `MAP_PRIVATE` mapping of the same offset gets its own copy on its own write. Until that write, reads share the cache page, because `do_read_fault` does not take this arm. Week 10's fork COW is the same copy applied to anonymous pages the parent already had. Here the flag is `VM_SHARED` being clear, and the copy is `copy_user_highpage` on the first write through this VMA.

5. Alignment, protection, `SIGBUS`, `msync`, `munmap`

Three alignments show up, and they are not the same rule.

The file offset passed to x86-64 `mmap` must be page-aligned, or the syscall returns `-EINVAL` before `ksys_mmap_pgoff`. `do_mmap` then page-aligns the length upward. A mapping of 100 bytes occupies a whole page in the address space. The extra bytes past `i_size`, still inside that page, are the ones `filemap_fault` refuses.

`msync` refuses a start address that is not page-aligned, and it refuses `MS_ASYNC` together with `MS_SYNC` (lines 42–47). The length is rounded up to a page. The comment at lines 18–31 says `MS_ASYNC` does not start I/O and does not mark pages dirty. Both of those used to happen. Dirty tracking is already done at fault time. The comment's advice is to call `fsync` or `fadvise(FADV_DONTNEED)` when the program actually wants writeback.

`MS_SYNC` on a shared file VMA is the path that actually waits. The syscall drops `mmap_lock`, calls `vfs_fsync_range` on the file offsets covered by the intersection of the range and the VMA, and continues with the next VMA. `MS_INVALIDATE` on a `VM_LOCKED` VMA returns `-EBUSY`. A hole in the range, where `find_vma` finds nothing or the VMA starts after the requested start, becomes `-ENOMEM` at the end through `unmapped_error`.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: mm/msync.c
 * lines: 78–104
 * function: msync
 */
        if ((flags & MS_INVALIDATE) &&
                (vma->vm_flags & VM_LOCKED)) {
            error = -EBUSY;
            goto out_unlock;
        }
        file = vma->vm_file;
        fstart = (start - vma->vm_start) +
             ((loff_t)vma->vm_pgoff << PAGE_SHIFT);
        fend = fstart + (min(end, vma->vm_end) - start) - 1;
        start = vma->vm_end;
        if ((flags & MS_SYNC) && file &&
                (vma->vm_flags & VM_SHARED)) {
            get_file(file);
            mmap_read_unlock(mm);
            error = vfs_fsync_range(file, fstart, fend, 1);
            fput(file);
            if (error || start >= end)
                goto out;
            mmap_read_lock(mm);
            vma = find_vma(mm, start);
        } else {
            if (start >= end) {
                error = 0;
                goto out_unlock;
            }
            vma = vma->vm_next;
        }
```

`MS_SYNC` does not include a private COW page. The condition requires `VM_SHARED`. A private store never entered the file, so there is nothing for `vfs_fsync_range` to flush on its behalf. Calling `msync` on a private mapping and expecting the file to change is a misunderstanding of section 4, not a property of the sync flag.

`munmap` removes the VMA. A later load of that address takes the `!vma` arm in `do_user_addr_fault` and does not reach `filemap_fault`. That fault is a segmentation fault. `SIGBUS` from this subsystem is the other outcome: the VMA is still there, and `filemap_fault` finds `offset >= max_off` because the file shrank, or it fails to make the page up to date and

returns `VM_FAULT_SIGBUS` at line 3055 after a `readpage` error. The access was a CPU instruction. There is no `errno`. Day 7 says what a handler is allowed to assume about the restarted instruction.

Inside a mapped page, the load or store uses the alignment the ISA requires for that instruction, not the page alignment `mmap` required for the offset. A one-byte load at `mapping + 1` faults on the same page as a load at `mapping + 0`, because `__handle_mm_fault` masks the address with `PAGE_MASK` before it builds the PTE. The byte offset inside the page is the low bits of the virtual address, applied by the CPU on the retry.

6. A shared page is not a lock

`MAP_SHARED` makes two processes' loads and stores hit one cache page. The hardware coherence protocol, the same one Week 11's false-sharing lesson depends on, is what makes a store on one core visible to a load on another. That is a statement about the memory system. It is not a statement about the C abstract machine.

If both sides are C programs and both access the same object, the program still needs a happens-before edge. Week 11's data-race rule does not grow an exception for "the bytes live in a mapping." Two non-atomic stores to the same `int` in a shared mapping, with no mutex and no atomic, are a data race. The mapping changed which physical page the stores hit. It did not insert synchronization.

A mutex in the mapping has to be process-shared when the waiters are different processes. A futex word in that page works for the same reason as the Week 11 futex: the kernel waits on an address that both tasks translate to the same page. Atomics from Week 11 Day 5 are the other tool. `msync` pushes dirty file data. It does not order one process's store against another's load. Threads already share the mapping because they share the page tables (Week 11 Day 1). `MAP_SHARED` across `fork` adds a second mm and a second set of PTEs aimed at the same pages. The synchronization requirement does not get smaller because there are two page tables.

7. The copy and the fault win different cases

A single sequential pass the program will not share is often cheaper as `read`. One syscall copies many pages, readahead runs either way, there is no user fault per file page, and EOF is a short count rather than a signal. Repeated or shared or random access is the fault path: the first touch pays `filemap_fault`, later touches are loads and stores, and a second `MAP_SHARED` mapping faults in the same cache page instead of taking its own copy from `read`. Bytes the program never touches are never copied.

A map, one touch per page, and `munmap` measures faults plus I/O plus teardown. A `read` into a buffer the program then parses measures the copy the mapping avoids, and a simpler failure mode. Quote the access pattern with the number, or the number does not say which mechanism won.

What both paths share is the page cache and the rule that the file's size is the limit of the bytes. `read` reports the limit as a short count. `mmap` reports a later violation of the limit as `SIGBUS` from `filemap_fault`. Choosing `mmap` to avoid checking the return value of `read` trades an errno for a signal on the instruction that walked off the file.

8. A mapping small enough to fault on purpose

```
long page = sysconf(_SC_PAGESIZE);
off_t off = 0;
size_t len = (size_t)page;
char *p;

if (off & (page - 1))
    return EINVAL;                 /* x86-64 mmap rejects this in-kernel */
p = mmap(NULL, len, PROT_READ | PROT_WRITE, MAP_SHARED, fd, off);
if (p == MAP_FAILED)
    return -1;
p[0] = 'A';                        /* may page-fault into the cache page */
if (msync(p, len, MS_SYNC) < 0)
    return -1;                     /* vfs_fsync_range on a VM_SHARED file */
if (munmap(p, len) < 0)
    return -1;
```

`mmap` returns without a PTE. `p[0] = 'A'` misses, traps, and `do_shared_fault` runs because the VMA is `VM_SHARED`. `filemap_fault` allocates a cache page. The store runs again and writes `'A'` into that page. `msync(MS_SYNC)` calls `vfs_fsync_range`. `munmap` drops the VMA, and a later load of `p[0]` faults with no VMA.

The same source with `MAP_PRIVATE` takes `do_cow_fault`. `'A'` lands in the copy. `msync` does not see `VM_SHARED`, so it does not sync that VMA, and the file still holds the original byte. `ftruncate` the file to zero while the shared mapping is live, then load `p[0]`. The VMA remains. `filemap_fault` returns `VM_FAULT_SIGBUS` because `offset >= max_off`. The load was not a syscall, so there is no errno. Day 7 is the safe description of that signal.

9. Mastery: Explain · Draw · Observe · Build

The exercise is ../challenges/day-006-mmap.sh. Predict which byte reaches the file under `MAP_SHARED`, and which byte stays in an anonymous copy under `MAP_PRIVATE`.

Explain

- Why `mmap` can return before any file byte has been copied, and which fault function performs the cache lookup.
- What `VM_SHARED` changes in `do_fault`, and what `copy_user_highpage` is copying onto.
- Why a load past the end of a still-mapped file is `SIGBUS` from `filemap_fault`, while a load of an address `munmap` removed never reaches `filemap_fault`.
- What `MS_SYNC` calls for a shared file VMA, and what `MS_ASYNC` does in this tree.
- Why two C programs storing into a `MAP_SHARED` page still need a mutex, a futex, or an atomic.

Draw

Draw the section 3 trace as a vertical strip: user load, TLB, trap, VMA, `filemap_fault`, page-cache page, PTE, retry. Put the file offset and `i_size` on the cache page. Beside it, draw the same file byte moving through `read`: syscall, cache page, `copy_to_user`, user buffer. Mark which path installs a PTE and which path returns a count.

Observe

Run the wrapper. Record `uname -r` and the tree id if you opened the sources. Note the mapping address, the store that faults, and whether a later `read` of the file observes the byte. If you truncate under a live mapping, record the signal and the fault address, and do not describe it as a failed `read`.

Build

Write a one-page account of one store to `MAP_SHARED` and the matching store to `MAP_PRIVATE`, using the functions from this tree: `ksys_mmap_pgoff`, `do_mmap`, `do_fault`, `filemap_fault`, `do_cow_fault`, `msync`. End with the sentence you will use when someone says mapping is faster: it removes a copy after the PTE exists, and it adds a fault, a TLB entry, and `SIGBUS` at the edge of the file.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Mmap

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 12-linux-io-and-readiness/challenges/day-006-mmap.sh
```

- View lab source on GitHub

Day 7 — I/O Debugging and the Comparison Report

Target time: approximately 3 hours

> A stuck `read`, a vanished writer, and a `SIGBUS` on a load are three different layers failing. The report names the layer that produced each fact, then walks one failure from the symptom to a regression check.

Days 4–6 put a byte queue behind two descriptors, a readiness wait in front of those descriptors, and a page-cache page behind a virtual address. None of those objects is a C variable the program can print. `/proc`, strace, and GDB ask the kernel, or the stopped task, to describe them. Each answer stops at a different layer. A transcript with no kernel identity and no binary identity is a story about a different machine.

The line-level claims below are from tree `c9b2385d6793`, a 5.10.134-based kernel. Before you repeat a line number, record `uname -r` and the tree id you actually opened. Week 10's report used the same rule for `task_struct`. It applies here to `struct file`, the pipe ring, and the VMA.

1. Why the descriptor table has to be asked for

The integer a program stores in `int fd` is an index. The `struct file`, the pipe's `head` and `tail`, and the VMA's file offset live in kernel memory. Week 9 is why a user load of that integer cannot observe them. The kernel builds a directory of symlinks at `/proc/PID/fd` and a text file per descriptor at `/proc/PID/fdinfo` when something reads them. Two reads are two generations. A task can `close` and `dup` between them.

`fs/proc/fd.c` generates the fdinfo text. `seq_show` locks the task, looks the descriptor up, and copies out the position and the flags.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/proc/fd.c
 * lines: 31–58
 * function: seq_show
 */
   task_lock(task);
   files = task->files;
   if (files) {
       unsigned int fd = proc_fd(m->private);

       spin_lock(&files->file_lock);
       file = files_lookup_fd_locked(files, fd);
       if (file) {
           struct fdtable *fdt = files_fdtable(files);

           f_flags = file->f_flags;
           if (close_on_exec(fd, fdt))
               f_flags |= O_CLOEXEC;

           get_file(file);
           ret = 0;
       }
       spin_unlock(&files->file_lock);
   }
   task_unlock(task);
   put_task_struct(task);

   if (ret)
       return ret;

   seq_printf(m, "pos:\t%lli\nflags:\t0%o\nmnt_id:\t%i\n",
          (long long)file->f_pos, f_flags,
          real_mount(file->f_path.mnt)->mnt_id);
```

`O_NONBLOCK` and `O_CLOEXEC` are visible here because they live in `f_flags` and the close-on-exec bitmap. A pipe that was supposed to be nonblocking, and whose fdinfo flags do not include the nonblocking bit, will block inside `pipe_read` on an edge-triggered drain. That is a Day 5 bug you can see without guessing.

After the position line, the same function optionally calls the file's own `show_fdinfo`. Pipes in this tree do not set that operation. `pipefifo_fops` has `read_iter`, `write_iter`, `poll`, and `release`, and no `show_fdinfo`. The `if` is false, and fdinfo for a pipe stops at position, flags, mount id, and whatever `show_fd_locks` printed.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/proc/fd.c
 * lines: 60–71
 * function: seq_show
 */
    /* show_fd_locks() never deferences files so a stale value is safe */
    show_fd_locks(m, file, files);
    if (seq_has_overflowed(m))
        goto out;

    if (file->f_op->show_fdinfo)
        file->f_op->show_fdinfo(m, file);

out:
    fput(file);
    return 0;
}
```

The symlink at `/proc/PID/fd/N` is not a stored path. `proc_fd_link` takes a reference on the task's file and copies `f_path`. `d_path` then renders it. For a pipe, the dentry operation `pipefs_dname` prints `pipe:[inode]`. Both ends of one pipe share that inode, so both symlinks print the same `pipe:[…]` text. That shared text is how you prove two descriptors are the two ends, or that a child inherited the end you thought you had closed.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/proc/fd.c
 * lines: 143–159
 * function: proc_fd_link
 */
static int proc_fd_link(struct dentry *dentry, struct path *path,
            struct task_struct *task)
{
    int ret = -ENOENT;
    unsigned int fd = proc_fd(d_inode(dentry));
    struct file *fd_file;

    fd_file = fget_task(task, fd);
    if (fd_file) {
        *path = fd_file->f_path;
        path_get(&fd_file->f_path);
        ret = 0;
        fput(fd_file);
    }

    return ret;
}
```

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: fs/pipe.c
 * lines: 855–866
 * function: pipefs_dname
 */
/*
 * pipefs_dname() is called from d_path().
 */
static char *pipefs_dname(struct dentry *dentry, char *buffer, int buflen)
{
    return dynamic_dname(dentry, buffer, buflen, "pipe:[%lu]",
                d_inode(dentry)->i_ino);
}

static const struct dentry_operations pipefs_dentry_operations = {
    .d_dname    = pipefs_dname,
};
```

`readlink` of the symlink is a syscall, like the status file in the Week 10 lesson. It shows the file that was installed at that index at the moment `fget_task` ran. It does not show `pipe->head`, the unread byte count, or whether a task is sleeping on `rd_wait`. Those facts need a trace, or `FIONREAD` on the pipe, which `pipe_ioctl` in `fs/pipe.c` computes by summing `buf->len` under the pipe mutex (lines 610–623). fdinfo's `pos` for a pipe is not that sum. `stream_open` gave the pipe no shared file position that a seek could use.

2. strace sees the syscall, not the ring

strace stops the task at syscall entry and exit through ptrace. The line it prints is the architecture's registers at that boundary: the number, the arguments, and the return value. Week 8 is which register holds the number on the educational path, and the same idea holds for the vDSO `syscall` instruction on a modern x86-64 process: the number and the arguments are in the ABI registers, and the return value comes back in the register the ABI uses for a function result. strace decodes those registers. It does not decode `pipe->writers`.

A `write` line that returns a positive count is Day 4's short-or-full copy. A `write` line that never returns is a task sleeping in `pipe_write`'s `wait_event_interruptible_exclusive` on `wr_wait`, which is backpressure, not a lost byte. An `epoll_wait` line that returns 1 is one ready event. The byte count is the next `read` line. Collapsing those two numbers is the Day 5 mistake, and strace is the tool that keeps them on separate lines.

What strace will not show is a load or a store. `mmap` appears. `msync` appears. The store that faults does not appear as a syscall. The next thing in the trace, if the fault is fatal, is the signal. That absence is evidence. A report that says "the read failed" when the trace shows no `read` after `mmap` has named the wrong layer.

GDB stops on the signal or on a breakpoint and can print the fault address and the instruction pointer. The instruction pointer is in the user program, at the load or store, which is the retry point from Day 6. GDB does not print `i_size` unless you have kernel symbols and a kernel target. For this week's userspace labs, GDB answers "which instruction faulted and at which address." The source answers "why that address is past the file."

3. Five results that get swapped

These are the failures the labs actually produce. Each one has a generator in the tree from Days 4–6.

`EBADF`. The descriptor index does not name a `struct file` in this task. `do_pipe2` never installed it, or `close` already released it, or the index was never returned. `ksys_mmap_pgoff` returns `-EBADF` when `fget` fails (`mm/mmap.c` lines 1699–1701). `do_epoll_wait` returns `-EBADF` when `fdget` on the epoll descriptor fails (`fs/eventpoll.c` lines 2345–2347). `do_epoll_ctl` does the same for the epoll fd and the target fd (lines 2159–2167), and returns `-EPERM` when the target file has no poll operation (lines 2169–2172). A trace that shows `epoll_ctl` failing with `EPERM` is a file that cannot be waited on, not a readiness bug. A trace that shows `EBADF` is a lifetime bug in the index: use after close, or the wrong integer.

`EPIPE` and `SIGPIPE`. Day 4's `pipe_write` posts both when `readers` is zero: `send_sig(SIGPIPE, current, 0)` and `-EPIPE` (`fs/pipe.c` lines 432–435). The default disposition kills the writer on the way back to user mode, so the C statement after `write` does not run and strace shows the signal rather than a handled errno. Ignoring or blocking `SIGPIPE` lets `write` return `-1` with `EPIPE`. A partial copy that then discovers the last reader has gone still posts the signal, and it keeps the positive count when some bytes were already written (lines 477–482). The report has to say which of those two returns it observed. "Broken pipe" as a shell phrase is not yet either of them.

`EAGAIN`. A nonblocking `pipe_read` or `pipe_write` returns `-EAGAIN` when the ring is empty or full and the other side still exists (Day 4, `pipe_read` lines 344–346 and `pipe_write` lines 547–550). `epoll_wait` timing out returns 0, not `EAGAIN`. `EAGAIN` from a nonblocking `read` during an `EPOLLET` drain means "the level is gone; go back to `epoll_wait`." Treating it as a fatal error drops the connection. Treating `epoll_wait`'s 0-timeout 0 as `EAGAIN` is the same confusion in the other direction.

epoll misuse. `EPOLL_CTL_ADD` of a key that is already present returns `-EEXIST`. `DEL` or `MOD` of a missing key returns `-ENOENT` (`do_epoll_ctl` lines 2266–2286). `EPOLLET` does not put the item back on `rdllist` (`ep_send_events_proc` lines 1764–1778), so a short read with no later wakeup leaves bytes in the pipe and the waiter asleep. `EPOLLONESHOT` clears the interest bits until `EPOLL_CTL_MOD`. Closing the last reference to a watched file runs `eventpoll_release` from `__fput` (Day 5). Closing a `dup` while another descriptor keeps the file does not. A stall whose strace shows one `read` and then `epoll_wait`, on a pipe whose `FIONREAD` is still nonzero, matches the edge-triggered arm. A stall whose fdinfo lacks `O_NONBLOCK` matches the drain that blocked inside `read` instead of returning `EAGAIN`.

`SIGBUS` on a mapped file. This is not an errno. Day 6's `filemap_fault` returns `VM_FAULT_SIGBUS` when the page index is past `i_size` (`mm/filemap.c` lines 2959–2961, and again under the page lock at lines 3022–3026). x86 turns that code into a signal:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: arch/x86/mm/fault.c
 * lines: 928–961
 * function: do_sigbus
 */
    /* Kernel mode? Handle exceptions or die: */
    if (!(error_code & X86_PF_USER)) {
        no_context(regs, error_code, address, SIGBUS, BUS_ADRERR);
        return;
    }

    /* User-space => ok to do another page fault: */
    if (is_prefetch(regs, error_code, address))
        return;

    sanitize_error_code(address, &error_code);

    if (fixup_vdso_exception(regs, X86_TRAP_PF, error_code, address))
        return;

    set_signal_archinfo(address, error_code);

#ifdef CONFIG_MEMORY_FAILURE
    if (fault & (VM_FAULT_HWPOISON|VM_FAULT_HWPOISON_LARGE)) {
        struct task_struct *tsk = current;
        unsigned lsb = 0;

        pr_err(
    "MCE: Killing %s:%d due to hardware memory corruption fault at %lx\n",
            tsk->comm, tsk->pid, address);
        if (fault & VM_FAULT_HWPOISON_LARGE)
            lsb = hstate_index_to_shift(VM_FAULT_GET_HINDEX(fault));
        if (fault & VM_FAULT_HWPOISON)
            lsb = PAGE_SHIFT;
        force_sig_mceerr(BUS_MCEERR_AR, (void __user *)address, lsb);
        return;
    }
#endif
    force_sig_fault(SIGBUS, BUS_ADRERR, (void __user *)address);
```

`mm_fault_error` calls `do_sigbus` when the fault code includes `VM_FAULT_SIGBUS` (lines 987–990). For a user-mode fault that is not a hardware-poison case, the signal is `SIGBUS` with `BUS_ADRERR` and the fault address. The default disposition terminates the process. A handler that returns resumes the faulting instruction, because a page fault returns to the instruction pointer the trap saved, which Day 6 traced. If the VMA is still there and the file is still short, the instruction faults again. A handler that returns without changing the mapping or the file size livelocks the task in that fault. The safe description for this week's labs is: record the address, treat the access as fatal, and do not return into the faulting instruction unless the program has already removed the cause, for example by `munmap` in a handler that then long-jumps to a recovery path it designed. Catching `SIGBUS` and ignoring it is not recovery. A load of an address that `munmap` already removed is a different signal: there is no VMA, and `do_user_addr_fault` takes `bad_area_nosemaphore` before `filemap_fault`.

4. One failure, all the way through

The shape is fixed. Symptom, hypothesis, evidence, root cause, fix, regression. The evidence column names its layer: source, API, trace, proc, or GDB.

Symptom. A producer process disappears while a consumer is starting. The shell reports that the producer died from `SIGPIPE`, or it reports nothing and `waitpid` shows death by that signal. No `EPIPE` is visible in the producer's own logging, because the producer has no logging after the `write` that killed it.

Hypothesis. `pipe_write` observed `readers == 0`. Either the consumer closed the read end, or the parent closed it before the child inherited a reference, or the child exited without reading. The write posted `SIGPIPE` and returned `-EPIPE` from the kernel, and the default disposition never let user code see the errno.

Evidence.

- API, predicted from Day 4: with `SIGPIPE` ignored, `write` returns `-1` and `errno == EPIPE`. With the default disposition, the process dies and that return is not observed in the program.
- Trace: `strace -f` shows `write` and then `SIGPIPE` for the default case, or `write(...) = -1 EPIPE` when the signal is ignored. There is no second `write` of the bytes that were supposed to follow.
- Proc, if you catch the process before it dies, or on the consumer: `/proc/PID/fd` for the consumer does not contain a `pipe:[inode]` matching the producer's write end, or the consumer has already exited and its `/proc` directory is gone. `ENOENT` on that directory is evidence the task is gone, not evidence about the pipe.
- Source: `fs/pipe.c` `pipe_write`, lines 432–435, tree `c9b2385d6793`. `send_sig(SIGPIPE, current, 0)` and `ret = -EPIPE`.
- GDB, optional: a breakpoint on `write` is too late to see the kernel check, and a catch of `SIGPIPE` shows the user instruction pointer still inside the libc `write` wrapper. That confirms the signal arrived on the way out of the syscall. It does not by itself show `readers`.

Root cause. The last reference on the read-end `struct file` was dropped before this `write`. `pipe_release` decremented `readers`. The producer's `write` is behaving as specified. The bug is the close order in the pipeline, the same missing `dup2`/`close` protocol as Day 4's section 4.

Fix. The parent keeps the read end open only until the child has inherited it, then closes the parent's copy. The child maps the read end onto descriptor 0 and closes the original indexes so it is not itself a writer. The producer ignores `SIGPIPE` only if a vanished consumer is a normal event, and then it treats `EPIPE` as "this connection is gone" rather than as a short write to retry blindly.

Regression. Two checks. A pipeline that writes three bytes and closes the write end makes the child `read` return 3 and then 0, and `waitpid` collects the child. A second program ignores `SIGPIPE`, closes the read end, writes one byte, and requires `errno == EPIPE` while the process is still alive. The first check guards the success path. The second guards the error path Day 4 separated from end-of-file. A test that only checks "the process exited" passes for both the bug and the fix.

5. The same shape for a readiness stall

Symptom. An edge-triggered epoll loop stops calling `read` while a writer has already pushed more bytes than one `read` accepted. The process sits in `epoll_wait`. A later write unsticks it. A quiet writer does not.

Hypothesis. `EPOLLET` did not requeue the item, and the program issued one `read` and went back to sleep. The pipe ring still holds bytes. No new `pipe_write` wakeup has run `ep_poll_callback`.

Evidence.

- Trace: `epoll_ctl` with `EPOLLET`, one `read` returning a positive count smaller than the write, then `epoll_wait` not returning. `ioctl(FIONREAD)` on the same pipe, from a debugger or a side path, returns a nonzero count. That ioctl is `pipe_ioctl` summing buffer lengths, not a guess.
- Proc: `/proc/PID/fdinfo/N` for the read end. If `O_NONBLOCK` is absent from `flags`, a different bug is also available: the next `read` can sleep inside the kernel instead of returning `EAGAIN`. The symlink still says `pipe:[inode]`.
- Source: `ep_send_events_proc` requeues only when `EPOLLET` is clear (`fs/eventpoll.c` lines 1764–1778). `pipe_read` returns `-EAGAIN` for a nonblocking empty ring (lines 344–346).
- API: `epoll_wait` returning 1 did not mean the pipe was empty afterwards. The following `read` is the byte count.

Root cause. The readiness bit was consumed as if it were a completion of the whole buffer. Edge trigger requires the caller to drain until `EAGAIN`.

Fix. Set `O_NONBLOCK` on the read end. After `epoll_wait` reports `EPOLLIN`, `read` until `EAGAIN`, and treat `0` as end-of-file. Do not treat `EAGAIN` as failure.

Regression. Write more than one `read` buffer, do not write again, and require the reader to observe every byte before it blocks in `epoll_wait`. A timeout of zero on a second `epoll_wait`, issued after a single short `read` and before the drain, must be allowed to return 0 on an edge-triggered set. That 0 is the bug's fingerprint. After the fix, that second wait is not reached until `EAGAIN`, and the byte count matches the write.

6. The same shape for `SIGBUS`

Symptom. A process that mapped a file dies with `SIGBUS`. strace shows `mmap` and `ftruncate` and then the signal. It does not show a failing `read`.

Hypothesis. The mapping still covers the address, and the file's `i_size` no longer does. `filemap_fault` returned `VM_FAULT_SIGBUS`.

Evidence.

- Trace: `mmap`, then `ftruncate` to a shorter length, then `SIGBUS`. No `read` syscall on the faulting thread.
- GDB: the instruction is a load or a store, and the fault address lies inside the mapping and at or beyond the new end of the file.
- Source: `filemap_fault` lines 2959–2961, and `do_sigbus` line 961, `force_sig_fault(SIGBUS, BUS_ADRERR, address)`, tree `c9b2385d6793`.

- API: there is no errno, because the faulting access was not a syscall. `msync` and `munmap` are the syscalls that could have been used before the load.

Root cause. The program kept using a mapping across a shrink of the file.

Fix. `munmap` before the access, or do not shrink a file that is still mapped, or bound the load to the size you sampled under a protocol that cannot race the truncate. A signal handler is not the fix unless it removes the VMA and does not return to the faulting instruction.

Regression. A load inside `i_size` succeeds and sees the file byte under `MAP_SHARED`. A load after `ftruncate` below that offset, with the mapping intact, is the expected `SIGBUS` and is not reported as a passing read. A load after `munmap` is a segmentation fault instead, which shows the two faults stayed distinct.

7. What each layer is allowed to prove

```
layer      answers                                      does not answer
source     which function returns -EPIPE, SIGBUS,       what this run did
           EAGAIN on tree c9b2385d6793
API        what this call returned, or which signal     why the ring was empty
           arrived, in this process
strace     the syscalls and the signal, with arguments  a load that never entered
           and return values                            the kernel as a syscall
/proc fd   the path text and the flags/pos at one read  head, tail, or the waiter
GDB        the faulting instruction and address        i_size, unless the target
                                                        is the kernel
```

A sentence in the report is done when it names one cell. "The writer died from `SIGPIPE`" is an API fact if `waitpid` decoded the signal, or a trace fact if strace printed it. "Because `readers` was zero" is a source fact, earned by the trace showing no live read end and by `pipe_write` being the function that posts that signal. Neither sentence alone is the root cause. The root cause is the close that dropped the last reader, which you earn by showing the missing descriptor in `/proc` or the `close` in the trace.

Optional tools stay optional. A ptrace policy that refuses strace or GDB belongs in the report as a refusal and the tool's diagnostic, the way Week 10 and Week 11 already treat a missing sanitizer. The refusal is not evidence that the pipe was empty. The source and the API checks still stand.

8. ToyOS, this tree, and version discipline

Week 8's pipe is a 64-byte linear buffer, descriptors 3 and 4, and a read that returns 0 as soon as the buffer is empty. There is no `writers` count, no `SIGPIPE`, no sleep on a full ring, and no `dup2` pipeline because there is no second process with its own fd table. Week 9's VFS names files with inodes and descriptor indexes, and it does not build `/proc`, a page cache, or epoll. ToyOS evidence for those labs is the serial line and QEMU GDB, not a proc symlink.

This tree's pipe is the object Days 4 and 5 read: a ring of pages, two `struct file` objects, `pipe_read` / `pipe_write`, and `epoll` hooked through `pipe_poll`. This tree's mapping is Day 6: `mmap` publishes a VMA, `filemap_fault` fills a PTE, `SIGBUS` fires when the index is past `i_size`. The comparison worth writing down is which invariant each system actually has.

```
question                         ToyOS Week 8–9              this Linux tree
where do pipe bytes live?       pipe_data[64]               pipe_buffer pages
empty, writer still open?       read returns 0              sleep, or -EAGAIN
last writer closed?             not represented             read returns 0
no reader?                       write fails the fit check   SIGPIPE and -EPIPE
many sources?                    one global pipe             select/poll/epoll
file bytes in user loads?       not implemented             mmap + filemap_fault
how do you observe it?          serial, QEMU GDB            proc, strace, GDB, source
```

Version discipline is part of the report, not a footer. `c9b2385d6793` is the tree these lessons quote. It is 5.10.134-based. The lab machine's `uname -r` may be a different release. `pipe_read`, `ep_send_events_proc`, `filemap_fault`, and `seq_show` move, split, and gain arguments. A line number copied from this lesson into a report about a different tree is a false citation. The honest citation is: the behavior I observed (return value, signal, proc text), the tree I opened, and the function I found there. If you did not open a tree, say so, and keep the claim at the API layer.

The same discipline applies to ToyOS. A serial line from the Week 8 image is evidence about that image. It is not evidence about `pipe_release`. Writing "ToyOS returns `EPIPE`" would be a false citation in the other direction: that image returns 0 on an empty buffer and has no `EPIPE`.

9. A report you can reread in a month

Record, in this order, before any interpretation:

```
uname -r
date
compiler and the lab binary's hash, if the wrapper prints one
tree id, if you opened sources (c9b2385d6793 for these notes)

for the failing run:
  the argv
  the wait status or the signal
  the strace lines you are willing to quote, or "strace refused: …"
  the /proc/PID/fd and fdinfo text, or "process already gone"
  the GDB instruction and fault address, or "GDB refused: …"

then the six headings:
  symptom
  hypothesis
  evidence   (each bullet tagged source / API / trace / proc / GDB)
  root cause
  fix
  regression (the command and the expected line)
```

The three cases in sections 4–6 are the templates. A new bug gets the same headings. A bug you have not localized yet stops at hypothesis and lists the evidence you still need. Filling in a root cause without a layer tag is how a short `read` becomes a fictional `EPIPE`.

The weekday's wrapper collects what it can collect without root and without tracing unrelated processes. A passing wrapper is not the report. The report is the six headings in your own words, tied to a tree id when you cite a function.

10. Mastery: Explain · Draw · Observe · Build

The exercise is ../challenges/day-007-io-debug-report.sh. Predict which of its checks are API assertions and which facts are left for the write-up.

Explain

- What `seq_show` prints for every descriptor, and why a pipe in this tree adds nothing through `show_fdinfo`.
- How `proc_fd_link` and `pipefs_dname` together make both ends of one pipe print the same `pipe:[inode]` text.
- Why `SIGPIPE` and `EPIPE` can describe one `write`, and which disposition hides the errno.
- Why a `SIGBUS` handler that returns, without changing the VMA or the file size, faults again on the same instruction.
- What ToyOS can show you about a pipe, and which of `EPIPE`, `epoll_wait`, and `SIGBUS` it cannot produce.

Draw

Draw one failing `write` with five boxes: the close that dropped the last reader, `pipe_release`, `pipe_write`, the signal disposition, and the strace line. Under it, draw a `SIGBUS` load: the store or load, the VMA, `filemap_fault`, `do_sigbus`, and the instruction pointer a handler would return to. Mark which boxes are source and which are trace.

Observe

Run the wrapper. Keep `uname -r`, the tree id if you opened sources, and one proc excerpt or an explicit "process already gone." Quote a kernel line only from a tree you identified. If strace or GDB is refused, keep the refusal text.

Build

Write one of sections 4, 5, or 6 as a one-page report with the six headings, each evidence bullet tagged with its layer. Include one ToyOS sentence and one version sentence: what the educational pipe does instead, and which tree the Linux function name came from. Leave the regression command in the write-up so the check can be run again without reconstructing it from memory.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Io Debug Report

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 12-linux-io-and-readiness/challenges/day-007-io-debug-report.sh
```

- View lab source on GitHub